Полный справочник к экзамену по программированию на C++ (Билеты 1–39)

Подробные ответы, таблицы, разбор типичных ошибок и компилируемые примеры кода по всем 39 вопросам экзаменационного перечня.

Оглавление

1. **Билет №1.** Определение и свойства алгоритма. Способы записи алгоритмов.
2. **Билет №2.** Основные этапы разработки и сопровождения программ.
3. **Билет №3.** Технологии программирования. Структурное программирование.
4. **Билет №4.** Классификация языков программирования. Способы описания языков программирования. Формы Бэкуса-Наура и синтаксические диаграммы.
5. **Билет №5.** Лексемы, выражения, операторы. Структура программы на языке C++. Типы данных, переменные, константы. Стандартные типы данных в C++. Приведение типов.
6. **Билет №6.** Переменные в C++. Область действия, область видимости и время жизни переменных.
7. **Билет №7.** Унарные и бинарные арифметические операторы в C++. Операторы сравнения. Операторы сдвига. Побитовые и логические операторы C++. Неполное вычисление логических выражений.
8. **Билет №8.** Операторы присваивания, инкремента и декремента. L-value и r-value выражения.
9. **Билет №9.** Условный оператор. Оператор запятая. Инструкция-выражение. Инструкции выбора if и switch. Инструкции передачи управления.
10. **Билет №10.** Организация циклов в C++. Цикл foreach. Ключевое слово auto.
11. **Билет №11.** Подпрограммы. Определение и объявление подпрограмм. Процедуры и функции.
12. **Билет №12.** Объявление и определение функции в C++. Формальные и фактические параметры. Соответствие типов в формальных и фактических параметрах. Способы передачи параметров (по значению и по адресу). Параметры по умолчанию. Встроенные функции. Стандартные функции. Побочный эффект подпрограмм.
13. **Билет №13.** Рекурсия. Формы рекурсивных подпрограмм. Глубина и текущий уровень рекурсии. Зацикливание рекурсивных подпрограмм. Примеры неэффективности

рекурсии.

14. **Билет №14.** Перегрузка функций. Ошибки, возникающие при перегрузке функций.
15. **Билет №15.** Указатели на функции. Callback-функции.
16. **Билет №16.** Макросы. Преимущества и недостатки использования макросов.
17. **Билет №17.** Шаблоны функций.
18. **Билет №18.** Директивы условной компиляции. Многофайловые проекты. Страж включения.
19. **Билет №19.** Массивы. Статические массивы.
20. **Билет №20.** Введение в STL, контейнеры vector и array.
21. **Билет №21.** Указатели. Типизированные и безтиповые указатели. Операция разыменования и операция получения адреса. Арифметические операции над указателями. Проблемы и типичные ошибки при работе с указателями. Константные указатели и указатели на константу.
22. **Билет №22.** Массивы. Реализация одномерных массивов с помощью указателей.
23. **Билет №23.** Многомерные массивы. Реализация многомерных массивов с помощью указателей. Динамическое выделение памяти под одномерные и двумерные массивы.
24. **Билет №24.** Указатели-параметры. Массивы указателей.
25. **Билет №25.** Передача одномерных массивов в качестве параметров. Передача двумерных массивов в качестве параметров.
26. **Билет №26.** Алгоритмы поиска и сортировки. Стандартные функции сортировки и поиска.
27. **Билет №27.** Строки. Операции над строками. Понятие нуль-терминированных строк.
28. **Билет №28.** Функция main. Передача параметров в функцию main.
29. **Билет №29.** Класс string. Функции стандартной библиотеки для обработки строк.
30. **Билет №30.** Пользовательские типы данных. Перечислимый тип enum. Тип struct. Массивы структур.
31. **Билет №31.** Типы ошибок и их обработка на этапах проектирования, трансляции, выполнения. Отладка программ в среде разработки. Степпинг, точки останова. Стек вызовов и отслеживание переменных.
32. **Билет №32.** Файлы. Основные принципы работы с файлами. Механизм чтения данных из файла. Определение конца файла. Открытие и закрытие файлов. Последовательный и прямой метод доступа.
33. **Билет №33.** Понятие потока ввода-вывода. Операции над потоками. Классификация потоков ввода-вывода. Иерархия потоковых классов.
34. **Билет №34.** Реализация форматированного ввода-вывода для потоковых классов. Флаги форматирования. Форматирующие методы. Манипуляторы потоков ввода-вывода и их использование.

35. **Билет №35.** Реализация неформатированного ввода-вывода для потоковых классов.
36. **Билет №36.** Текстовые файлы. Форматированный ввод-вывод.
37. **Билет №37.** Бинарные файлы. Последовательный метод доступа.
38. **Билет №38.** Организация прямого доступа к файлу.
39. **Билет №39.** Исключительные ситуации. Классификация исключительных ситуаций. Операторы для работы с исключениями. Защищённые блоки. Выброс системных и пользовательских исключений. Последовательность действий при обработке исключений. Необработанные исключения. Собственная функция аварийного завершения.

БИЛЕТ №1 · АЛГОРИТМЫ И ОП

1. Определение и свойства алгоритма. Способы записи алгоритмов.

Алгоритм — это точное, конечное и однозначное предписание исполнителю совершить последовательность действий для получения решения задачи за конечное время.

Ключевые тезисы:

- Дискретность, детерминированность, результативность, конечность и массовость — 5 фундаментальных свойств алгоритма.
- Основные способы записи: словесный, псевдокод, графический (блок-схемы по ГОСТ 19.701-90) и программный (код на ЯП).
- Любой корректный алгоритм должен завершаться за конечное число шагов при любых допустимых входных данных.

Понятие и фундаментальные свойства алгоритма

Алгоритм — это строго определённая последовательность элементарных действий (шагов), преобразующая исходные данные в искомый результат за конечное число шагов.

Исполнитель алгоритма — это субъект или техническое устройство (процессор, виртуальная машина, человек), способное однозначно интерпретировать и выполнять команды алгоритма без необходимости домысливания.

- Дискретность (прерывность): процесс решения задачи разбит на отдельные элементарные шаги (команды). Переход к следующему шагу возможен только после полного завершения предыдущего.
- Детерминированность (определённость, однозначность): каждый шаг алгоритма строго однозначен и не допускает двусмысленного толкования. При одних и тех же входных данных алгоритм всегда выдаёт одинаковый результат.
- Конечность (финитность): алгоритм обязан завершать свою работу за конечное число шагов за конечный промежуток времени.

- Результативность (направленность): по завершении работы алгоритма обязательно должен быть получен искомый результат (либо явное сообщение о невозможности решения задачи при заданных параметрах).
- Массовость (универсальность): алгоритм разрабатывается не для одного частного набора чисел, а для целого класса однотипных задач, различающихся входными данными.

Способы записи алгоритмов

На практике при проектировании программного обеспечения используются четыре основных способа представления алгоритмов, от неформальных к строго формализованным:

Способ записи	Характеристика	Преимущества и применение
Словесный (вербальный)	Описание шагов на естественном языке (русском, английском).	Прост для первичного понимания, но страдает многословностью и неоднозначностью.
Псевдокод	Полуформализованный язык, сочетающий ключевые конструкции ЯП (если, пока, для) и математическую нотацию.	Не привязан к конкретному компилятору; идеален для учебников и архитектурного проектирования.
Графический (блок-схемы)	Система геометрических фигур (ГОСТ 19.701-90 / ISO 5807): овал (начало/конец), параллелограмм (ввод/вывод), ромб (условие), прямоугольник (действие).	Максимальная наглядность структуры ветвлений и циклов для небольших модулей.
Программный (текст на ЯП)	Запись по строгим синтаксическим и семантическим правилам языка программирования (например, C++).	Готов к непосредственной трансляции и исполнению на компьютере.

Пример кода C++: Алгоритм Евклида (нахождение НОД) — от словесного описания к C++

Словесный алгоритм: Пока второе число b не равно 0, заменять пару (a, b) на (b, a mod b). Когда b станет равно 0, число a — искомый наибольший общий делитель.

```
#include <iostream>

// Программная запись алгоритма Евклида
int gcd(int a, int b) {
    // Свойство конечности: на каждом шаге остаток строго убывает (b_new
    while (b != 0) {
        int remainder = a % b; // Элементарный дискретный шаг
        a = b;
        b = remainder;
    }
    return a; // Результативность: возврат НОД
}

int main() {
    int x = 1071, y = 462;
    std::cout << "НОД(" << x << ", " << y << ") = " << gcd(x, y) << std:
    return 0;
}
```

Результат выполнения (stdout):

```
НОД(1071, 462) = 21
```

Совет для экзамена: На экзамене часто спрашивают разницу между конечностью и результативностью: конечность гарантирует остановку процесса за конечное число шагов, а результативность — получение осмысленного ответа при остановке.

БИЛЕТ №2 · АЛГОРИТМЫ И ОП

2. Основные этапы разработки и сопровождения программ.

Жизненный цикл ПО включает анализ требований, проектирование архитектуры и алгоритмов, кодирование, трансляцию, тестирование, отладку и длительное сопровождение.

Ключевые тезисы:

- Постановка задачи и ТЗ определяют «что должна делать программа», а проектирование — «как она это сделает».
- Трансляция в C++ состоит из препроцессинга, компиляции в объектный код (.obj / .o) и компоновки (линковки) в исполняемый файл.
- Тестирование обнаруживает наличие ошибок, а отладка (debugging) находит их причину в коде и устраняет её.

Последовательность этапов жизненного цикла программы

Промышленная и учебная разработка программного обеспечения представляет собой инженерный процесс, разделённый на взаимосвязанные фазы:

- 1. Постановка задачи и анализ требований: формулировка цели, входных и выходных данных, ограничений по памяти и времени, составление Технического Задания (ТЗ).
- 2. Проектирование и алгоритмизация: декомпозиция системы на модули и подпрограммы, выбор структур данных и разработка алгоритмов (псевдокод, диаграммы).
- 3. Кодирование (реализация): перевод спроектированных алгоритмов и структур данных в исходный код на языке программирования (файлы .cpp и .h).
- 4. Трансляция (препроцессинг, компиляция и компоновка): перевод исходного текста в машинный код целевой платформы и сборка единого исполняемого модуля.

- 5. Тестирование и отладка: проверка программы на наборе тестовых сценариев (граничные, штатные, некорректные данные) и локализация обнаруженных дефектов.
- 6. Документирование и ввод в эксплуатацию: подготовка руководства пользователя, технической документации API и развёртывание продукта.
- 7. Сопровождение (Maintenance): исправление выявленных у пользователей ошибок (корректирующее), адаптация под новые ОС/оборудование (адаптивное) и расширение функциональности (совершенствующее).

Этапы трансляции программы на языке C++

Понимание конвейера сборки C++ критически важно для диагностики ошибок компиляции (Syntax/Type Errors) и ошибок компоновщика (Linker Errors):

Фаза сборки	Инструмент	Вход → Выход	Выполняемые действия
1. Препроцессинг	Препроцессор (cpp)	main.cpp → main.i	Обработка директив #include, подстановка макросов #define, условная компиляция #ifdef, удаление комментариев.
2. Компиляция	Компилятор (g++, clang, cl)	main.i → main.o (или .obj)	Лексический, синтаксический и семантический анализ, оптимизация кода, генерация машинного объектного кода.
3. Компоновка (Линковка)	Компоновщик (ld / link)	main.o + lib.a → app.exe	Связывание вызовов функций между разными объектными файлами и подключение стандартных библиотек C++.

Пример кода C++: Защитное программирование и модульное тестирование на этапе разработки

Пример разделения бизнес-логики (вычисление квадратного корня по методу Ньютона) и набора автоматических проверок (тестирование граничных случаев).

```
#include <iostream>
#include <cmath>
#include <cassert>

// Этап кодирования: чистая функция с проверкой входных данных
double safeSqrtNewton(double x, double eps = 1e-7) {
    if (x < 0.0) return -1.0; // Код ошибки для отрицательных чисел
    if (x == 0.0) return 0.0; // Граничный случай
    double guess = x;
    while (std::abs(guess * guess - x) > eps) {
        guess = 0.5 * (guess + x / guess);
    }
    return guess;
}

int main() {
    // Этап тестирования: проверка штатных и граничных значений
    assert(safeSqrtNewton(0.0) == 0.0);
    assert(safeSqrtNewton(-5.0) == -1.0);
    assert(std::abs(safeSqrtNewton(25.0) - 5.0) < 1e-6);

    std::cout << "Все тесты успешно пройдены! sqrt(25) = "
              << safeSqrtNewton(25.0) << std::endl;
    return 0;
}
```

Результат выполнения (stdout):

```
Все тесты успешно пройдены! sqrt(25) = 5
```

Совет для экзамена: Обязательно различайте тестирование и отладку: тестирование отвечает на вопрос «Есть ли в программе ошибка?», а отладка — «На какой строке и почему она возникает?».

БИЛЕТ №3 · АЛГОРИТМЫ И ОП

3. Технологии программирования. Структурное программирование.

Структурное программирование (Дейкстра, Бём, Якопини) — методология разработки ПО, основанная на отказе от безусловного перехода goto, использовании трёх базовых управляющих конструкций и проектировании «сверху вниз».

Ключевые тезисы:

- Теорема Бёма — Якопини: любой алгоритм можно выразить с помощью трёх базовых структур: следование, ветвление и цикл.
- Отказ от оператора goto предотвращает появление «спагетти-кода» и делает поток управления предсказуемым.
- Проектирование «сверху вниз» (Top-Down Design) и модульная декомпозиция разбивают сложную задачу на иерархию простых подпрограмм.

Эволюция технологий программирования

По мере роста сложности программных систем подходы к организации кода прошли несколько ключевых парадигм:

- Неструктурное (монолитное) программирование (1950–1960-е): программа писалась как единый список команд с обильным использованием переходов goto. Приводило к нечитаемому «спагетти-коду».
- Процедурное и структурное программирование (конец 1960-х — 1970-е): Эдсгер Дейкстра, Никлаус Вирт. Декомпозиция на функции/процедуры и использование только структурированных управляющих конструкций.
- Модульное программирование: группировка логически связанных функций и данных в отдельные независимые файлы-модули со скрытием внутренней реализации и открытым интерфейсом.
- Объектно-ориентированное программирование (ООП): объединение данных (полей) и алгоритмов их обработки (методов) в единые сущности — классы и

объекты (инкапсуляция, наследование, полиморфизм).
- Обобщённое программирование (Generic Programming): написание алгоритмов и структур данных, параметризованных типами (шаблоны template в C++ и библиотека STL).

Принципы структурного программирования и теорема Бёма — Якопини

В 1966 году Коррадо Бём и Джузеппе Якопини доказали фундаментальную теорему, а в 1968 году Эдсгер Дейкстра опубликовал знаменитую статью «Оператор Go To считается вредным».

- 1. Базис из трёх управляющих конструкций: любой вычислительный алгоритм любой сложности может быть построен исключительно из трёх структур:
- • Следование (композиция): последовательное выполнение операторов один за другим ({ S1; S2; }).
- • Ветвление (альтернатива): выбор одного из путей выполнения в зависимости от условия (if-else, switch).
- • Цикл (итерация): многократное повторение блока команд, пока истинно условие (while, do-while, for).
- 2. Правило одного входа и одного выхода: каждая базовая конструкция и каждая подпрограмма должна иметь ровно одну точку входа и одну логическую точку выхода.
- 3. Проектирование «сверху вниз» (метод пошаговой детализации Вирта): задача сначала описывается на самом высоком уровне абстракции через вызовы подзадач, после чего каждая подзадача последовательно детализируется.

Пример кода C++: Сравнение неструктурного кода (goto) и структурного подхода

Замена запутанного цикла с метками и goto на чистую комбинацию базовых конструкций цикла (for) и ветвления (if) внутри выделенной подпрограммы.

```
#include <iostream>

// Структурный подход: декомпозиция в функцию с одним входом и одним вых
int sumPositiveElements(const int arr[], int size) {
    int sum = 0;                // 1. Следование (инициализация)
```

```
    for (int i = 0; i < size; ++i) { // 2. Цикл (повторение)
        if (arr[i] > 0) {        // 3. Ветвление (выбор)
            sum += arr[i];
        }
    }
    return sum;
}

int main() {
    int data[] = {12, -5, 8, 0, -3, 15};
    int n = sizeof(data) / sizeof(data[0]);
    std::cout << "Сумма положительных элементов: "
              << sumPositiveElements(data, n) << std::endl;
    return 0;
}
```

Результат выполнения (stdout):

```
Сумма положительных элементов: 35
```

Совет для экзамена: Если на экзамене попросят назвать 3 кита структурного программирования: 1) три базовые конструкции (следование, ветвление, цикл), 2) отказ от goto, 3) проектирование сверху вниз (декомпозиция на подпрограммы).

БИЛЕТ №4 · АЛГОРИТМЫ И ОП

4. Классификация языков программирования. Способы описания языков программирования. Формы Бэкуса-Наура и синтаксические диаграммы.

Языки программирования классифицируются по уровню близости к железу, способу трансляции и парадигме. Для формального описания синтаксиса используются Формы Бэкуса — Наура (БНФ/РБНФ) и синтаксические диаграммы Вирта.

Ключевые тезисы:

- Каждый язык определяется тремя аспектами: синтаксисом (правила записи), семантикой (смысл конструкций) и прагматикой (практическое применение).
- В БНФ нетерминальные символы заключаются в угловые скобки <...>, символ ::= означает «определяется как», а черта | означает альтернативу «ИЛИ».
- Синтаксические диаграммы графически представляют те же правила: прямоугольники — нетерминалы, скруглённые блоки/овалы — терминалы.

Классификация языков программирования

Языки программирования принято классифицировать по трём независимым критериям:

Критерий	Категории языков	Примеры и особенности
По уровню абстракции	Низкого уровня / Высокого уровня	Низкий: Машинный код, Assembler (привязаны к архитектуре CPU). Высокий: C++, Java, Python (платформонезависимый синтаксис). C/C++ часто называют языками

Критерий	Категории языков	Примеры и особенности
		среднего уровня за прямую работу с памятью.
По способу исполнения	Компилируемые / Интерпретируемые / Байткод (JIT)	Компилируемые (C, C++, Rust) — транслируются целиком в машинный код до запуска. Интерпретируемые (Python, JS) — выполняются построчно. Гибридные (Java, C#) — компилируются в байт-код ВМ.
По парадигме	Императивные (процедурные, ООП) и Декларативные (функциональные, логические)	Императивные (C, C++, Pascal) описывают КАК достичь цели через изменение состояния памяти. Декларативные (SQL, Haskell, Prolog) описывают ЧТО нужно получить.

Формальное описание языков: БНФ и синтаксические диаграммы

Естественный язык слишком неоднозначен для создания компиляторов. Поэтому синтаксис языка задаётся строго математически через метаязык Форм Бэкуса — Наура (BNF — Backus-Naur Form) или Расширенных БНФ (EBNF).

В БНФ используются: Терминалы (конечные символы алфавита языка: цифры, буквы, знаки +, ;), Нетерминалы (вспомогательные синтаксические категории в угловых скобках <идентификатор>) и Метасимволы (::= — определение, | — альтернатива, [...] — необязательный элемент в РБНФ, {...} — повторение 0 и более раз в РБНФ).

Синтаксические диаграммы (диаграммы Вирта) — это графический эквивалент БНФ, где терминалы изображаются в кругах или овалах, нетерминалы — в прямоугольниках, а стрелки показывают допустимые пути чтения конструкции.

Пример кода C++: Описание идентификатора и целого числа в БНФ и проверка в C++

Формальная запись правила идентификатора C++ в БНФ и программный лексический анализатор, проверяющий строку по этому правилу.

```
/*
  Запись правил в форме Бэкуса-Наура (БНФ):
  <буква> ::= a | b | ... | z | A | B | ... | Z | _
  <цифра> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
  <идентификатор> ::= <буква> | <идентификатор><буква> | <идентификатор>

  В Расширенной БНФ (РБНФ):
  <идентификатор> ::= <буква> { <буква> | <цифра> }
*/
#include <iostream>
#include <string>
#include <cctype>

bool isValidIdentifier(const std::string& s) {
    if (s.empty()) return false;
    // Первый символ обязан быть <буквой> или '_'
    if (!(std::isalpha(static_cast<unsigned char>(s[0])) || s[0] == '_')
        return false;
    }
    // Последующие символы: { <буква> | <цифра> }
    for (size_t i = 1; i < s.size(); ++i) {
        char c = s[i];
        if (!(std::isalnum(static_cast<unsigned char>(c)) || c == '_'))
            return false;
        }
    }
    return true;
}

int main() {
    std::cout << std::boolalpha;
    std::cout << "my_var1: " << isValidIdentifier("my_var1") << "
";
    std::cout << "1st_var: " << isValidIdentifier("1st_var") << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
my_var1: true
1st_var: false
```

Совет для экзамена: На экзамене часто просят написать БНФ для целого числа со знаком или вещественного числа. Запомните рекурсивный приём: <целое_без_знака> ::= <цифра> |

<целое_без_знака><цифра>, а <целое> ::= <целое_без_знака> | +<целое_без_знака> | -<целое_без_знака>.

БИЛЕТ №5 · ТИПЫ И ОПЕРАТОРЫ

5. Лексемы, выражения, операторы. Структура программы на языке C++. Типы данных, переменные, константы. Стандартные типы данных в C++. Приведение типов.

Программа на C++ состоит из директив препроцессора, объявлений и функции main(). Минимальные единицы текста — лексемы, из которых строятся выражения и инструкции. Система типов строго контролирует размер данных и правила их неявного и явного преобразования.

Ключевые тезисы:

- 6 видов лексем C++: ключевые слова, идентификаторы, литералы (константы), операторы, разделители (пунктуаторы) и комментарии (удаляются препроцессором).
- Фундаментальные типы: целочисленные (int, short, long, long long), символьные (char, wchar_t), логический (bool), вещественные (float, double, long double) и пустой тип (void).
- В современном C++ вместо C-style приведения (int)x рекомендуется использовать именованные операторы static_cast<T>(x), const_cast, reinterpret_cast.

Лексемы, выражения, операторы и структура программы

Лексема (токен) — минимальная неделимая смысловая единица языка, выделяемая компилятором на этапе лексического анализа. К лексемам относятся: ключевые слова (if, return, int, const), идентификаторы (имена переменных и функций), литералы (42, 3.14, 'A', "text"), знаки операций (+, ==, &&) и пунктуаторы (; {} () ,).

Выражение (Expression) — синтаксическая конструкция из операндов (переменных, литералов, вызовов функций) и операторов, которая вычисляет некоторое значение определённого типа (например, a + b * 2).

Инструкция / Оператор языка (Statement) — законченное предписание компилятору выполнить действие (завершается точкой с запятой ; или образует составной блок { ... }).

Стандартные типы данных, константы и приведение типов

Тип данных определяет: 1) объём выделяемой памяти в байтах, 2) способ двоичного кодирования данных, 3) диапазон допустимых значений и 4) набор допустимых операций.

Константы в C++ создаются с помощью квалификатора const (значение фиксируется при инициализации) или constexpr (вычисляется гарантированно на этапе компиляции).

- Неявное приведение (Implicit conversion): выполняется компилятором автоматически по правилам арифметического расширения (Integral Promotion: char/short → int → long long → double), чтобы избежать потери точности.
- Сужающее преобразование (Narrowing): присваивание double в int отбрасывает дробную часть (3.99 → 3).
- Явное приведение (Explicit cast): C-стиль (double)a, функциональный стиль double(a), и безопасный стандартный C++ стиль static_cast<double>(a).

Тип данных	**Размер (обычно x64)**	**Диапазон / Точность**	**Назначение**
bool	1 байт	false (0) или true (1)	Логические значения
char / unsigned char	1 байт (8 бит)	-128..127 / 0..255	Символы ASCII и однобайтовые целые
short / int / long long	2 / 4 / 8 байт	int: около ±2.14 млрд	Целые числа со знаком (или unsigned без знака)
float / double	4 / 8 байт (IEEE 754)	7 знаков / 15–16 знаков мантиссы	Вещественные числа с плавающей точкой
void	0 (не имеет значений)	—	Отсутствие возвращаемого значения или базовый тип для void*

Пример кода C++: Типы данных, константы и неявное/явное приведение типов

Демонстрация целочисленного деления, расширения типов и безопасного приведения через static_cast.

```
#include <iostream>

int main() {
    constexpr double PI = 3.1415926535; // Константа времени компиляции
    int numerator = 7;
    int denominator = 2;

    // Ошибка новичка: целочисленное деление до присваивания в double
    double wrongDiv = numerator / denominator; // 7 / 2 = 3 -> 3.0

    // Явное приведение типа через static_cast<double>
    double exactDiv = static_cast<double>(numerator) / denominator; // 7

    // Сужающее приведение (отбрасывание дробной части)
    int truncatedPi = static_cast<int>(PI);

    std::cout << "wrongDiv = " << wrongDiv << "
";
    std::cout << "exactDiv = " << exactDiv << "
";
    std::cout << "truncatedPi = " << truncatedPi << "
";
    std::cout << "sizeof(int) = " << sizeof(int) << " bytes, sizeof(doub
";
    return 0;
}
```

Результат выполнения (stdout):

```
wrongDiv = 3
exactDiv = 3.5
truncatedPi = 3
sizeof(int) = 4 bytes, sizeof(double) = 8 bytes
```

Совет для экзамена: Внимательно следите за выражением вида double r = 1 / 2 * 5.0; Так как 1 и 2 — целые литералы типа int, операция 1 / 2 выполнится первой по правилам целочисленного деления и даст 0, поэтому всё выражение станет равно 0.0!

БИЛЕТ №6 · ТИПЫ И ОПЕРАТОРЫ

6. Переменные в C++. Область действия, область видимости и время жизни переменных.

Переменная — это именованная область памяти определённого типа. Её поведение в программе определяется тремя характеристиками: областью действия (где объект существует логически), областью видимости (где к нему можно обратиться по имени) и временем жизни (когда память выделена физически).

Ключевые тезисы:

- Локальные (автоматические) переменные размещаются в стеке и уничтожаются при выходе из блока { }. Неинициализированные локальные переменные содержат «мусор».
- Глобальные и локальные статические (static) переменные живут на протяжении всей работы программы и по умолчанию инициализируются нулями.
- Если локальная переменная перекрывает (затеняет) глобальную с тем же именем, обратиться к глобальной можно через унарный оператор разрешения области видимости ::name.

Область действия, область видимости и время жизни

Часто студенты путают область действия (Scope) и область видимости (Visibility), а также время жизни (Storage Duration / Lifetime):

- Область действия (Scope): участок текста программы, в пределах которого объявление идентификатора активно (блочная/локальная область внутри { ... }, область функции, область пространства имён/глобальная область файла, область класса).
- Область видимости (Visibility): часть области действия, в которой к переменной можно непосредственно обратиться по её простому имени. Если во вложенном блоке объявить переменную с тем же именем, внешняя

переменная остаётся в своей области действия, но временно теряет видимость (происходит затенение — shadowing).

- Время жизни (Lifetime / Storage Duration): интервал времени исполнения программы, в течение которого за переменной закреплена физическая ячейка оперативной памяти.

Классы памяти и категории переменных в C++

В зависимости от места объявления и спецификаторов (static, extern, thread_local) переменные делятся на следующие категории:

Вид переменной	Где хранится	Время жизни	Значение по умолчанию
Локальная (автоматическая)	Стек вызовов (Stack)	От входа в блок { } до выхода из него	Не определено (случайный «мусор» в памяти)
Глобальная	Сегмент данных (.data / .bss)	Всё время работы программы (до входа в main и после выхода)	Автоматически обнуляется (0, 0.0, nullptr)
Локальная статическая (static)	Сегмент данных (.data / .bss)	Всё время работы программы (инициализируется 1 раз при первом проходе)	Автоматически обнуляется; видима только внутри своей функции
Динамическая (new / delete)	Куча (Heap / Free Store)	От вызова оператора new до явного вызова delete	Без скобок new int — мусор; со скобками new int() — 0

Пример кода C++: Затенение имён (::) и сохранение состояния в static-переменной

Показывает работу глобальной переменной, затенение её локальной переменной во вложенном блоке и подсчёт вызовов функции через локальную static переменную.

```
#include <iostream>

int counter = 100; // Глобальная переменная (время жизни: вся программа)
```

```
void trackCalls() {
    // Инициализируется только при ПЕРВОМ вызове функции, но живёт до ко
    static int callCount = 0;
    int localTemp = 0; // Создаётся заново при каждом вызове в стеке
    ++callCount;
    ++localTemp;
    std::cout << "Вызов #" << callCount << " (localTemp = " << localTemp
";
}

int main() {
    int counter = 50; // Локальная переменная затеняет глобальную ::coun
    {
        int counter = 10; // Внутренний блок затеняет обе внешние переме
        std::cout << "Внутренний counter = " << counter << "
";
        std::cout << "Глобальный ::counter = " << ::counter << "
";
    }
    std::cout << "Локальный в main counter = " << counter << "
";

    trackCalls();
    trackCalls();
    trackCalls();
    return 0;
}
```

Результат выполнения (stdout):

```
Внутренний counter = 10
Глобальный ::counter = 100
Локальный в main counter = 50
Вызов #1 (localTemp = 1)
Вызов #2 (localTemp = 1)
Вызов #3 (localTemp = 1)
```

Совет для экзамена: Локальная переменная с ключевым словом static сочетает в себе глобальное время жизни (значение не теряется между вызовами функции) и локальную область видимости (никто снаружи функции не может её испортить).

БИЛЕТ №7 · ТИПЫ И ОПЕРАТОРЫ

7. Унарные и бинарные арифметические операторы в C++. Операторы сравнения. Операторы сдвига. Побитовые и логические операторы C++. Неполное вычисление логических выражений.

Операторы C++ делятся на арифметические, реляционные (сравнения), логические (работают с bool и используют короткую схему вычисления) и побитовые (работают с отдельными разрядами целых чисел).

Ключевые тезисы:

- Оператор остатка от деления % применим ТОЛЬКО к целочисленным типам (для double используется функция std::fmod из <cmath>).
- Побитовые операторы (&, |, ^, ~, <<, >>) обрабатывают каждый бит числа независимо; сдвиг влево x << k эквивалентен умножению на 2^k.
- Неполное (короткое) вычисление (Short-circuit evaluation): в A && B при ложном A правая часть B НЕ вычисляется; в A || B при истинном A правая часть B НЕ вычисляется.

Арифметические операторы, операторы сравнения и логические операторы

Унарные арифметические операторы (+x, -x) имеют более высокий приоритет, чем бинарные мультипликативные (*, /, %) и аддитивные (+, -). При делении двух целых чисел (int / int) дробная часть отбрасывается в сторону нуля.

Операторы сравнения (==, !=, <, >, <=, >=) возвращают значение типа bool (true или false). В C++20 также добавлен оператор трёхстороннего сравнения <=>.

Логические операторы: унарное отрицание ! (НЕ), бинарная конъюнкция && (логическое И) и бинарная дизъюнкция || (логическое ИЛИ). Приоритет: сначала !, затем &&, и ниже всего ||.

Побитовые операторы и операторы битового сдвига

Побитовые операции применяются только к целочисленным типам (int, unsigned int, char, long long) и воздействуют непосредственно на двоичный код числа:

Оператор	Название	Действие над битами	Типичное применение
&	Побитовое И (AND)	1 только если оба бита равны 1	Проверка бита или сброс битов по маске (x & mask)
|	Побитовое ИЛИ (OR)	1 если хотя бы один бит равен 1	Установка нужного бита в 1 (x | (1 << k))
^	Исключающее ИЛИ (XOR)	1 если биты РАЗНЫЕ (1^0=1, 1^1=0)	Инверсия выбранных битов, быстрый обмен без временной переменной
~	Побитовое НЕ (NOT)	Инвертирует все биты (0↔1)	Создание инверсных масок: x & ~(1 << k) сбрасывает k-й бит
<<	Сдвиг влево	Сдвигает биты к старшим, справа заполняет нулями	Быстрое умножение на степень двойки: x << k равно x * 2^k
>>	Сдвиг вправо	Сдвигает биты к младшим разрядам	Быстрое целочисленное деление на 2^k (для неотрицательных)

Неполное (короткое) вычисление логических выражений (Short-circuit evaluation)

В отличие от побитовых & и |, которые всегда вычисляют оба своих операнда, логические операторы && и || вычисляют операнды строго слева направо и прекращают вычисление, как только итоговый результат становится очевиден:

- В выражении (expr1 && expr2): если expr1 оказалось false, то всё И-выражение гарантированно false. Поэтому expr2 ВООБЩЕ НЕ ВЫЧИСЛЯЕТСЯ (даже если там вызов функции или инкремент ++x).
- В выражении (expr1 || expr2): если expr1 оказалось true, то всё ИЛИ-выражение гарантированно true. Поэтому expr2 НЕ ВЫЧИСЛЯЕТСЯ.

- Практическая польза: защита от деления на ноль (denom != 0 && num / denom > 2) и защита от разыменования нулевого указателя (ptr != nullptr && *ptr > 0).

Пример кода C++: Битовые маски и эффект короткого вычисления (Short-circuit)

Управление битовыми флагами числа и демонстрация того, как правый операнд оператора && пропускается при ложном левом операнде.

```
#include <iostream>

int main() {
    // 1. Работа с битами числа
    unsigned int flags = 0b0010; // 2 в десятичной системе (установлен б
    flags |= (1u << 3);          // Устанавливаем бит №3 -> 0b1010 (10)
    bool isBit1Set = (flags & (1u << 1)) != 0; // Проверяем бит №1
    std::cout << "flags = " << flags << ", бит №1 установлен: " << std::
";

    // 2. Короткое вычисление логических выражений
    int a = 0;
    int counter = 10;

    // Так как (a != 0) равно false, выражение (++counter > 0) НЕ ВЫПОЛН
    if (a != 0 && ++counter > 0) {
        std::cout << "Внутри if
";
    }
    std::cout << "counter после && остался равен: " << counter << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
flags = 10, бит №1 установлен: true
counter после && остался равен: 10
```

Совет для экзамена: Любимая задача на экзамене: int x = 0, y = 5; if (x == 0 || ++y > 0) { ... } Чему равен y? Ответ: y останется равен 5, так как левая часть оператора || истинна (x == 0), и правая часть ++y даже не запускалась!

БИЛЕТ №8 · ТИПЫ И ОПЕРАТОРЫ

8. Операторы присваивания, инкремента и декремента. L-value и r-value выражения.

Операторы присваивания имеют правую ассоциативность и требуют изменяемое l-value в качестве левого операнда. Префиксный инкремент (++x) возвращает изменённый объект (l-value), а постфиксный (x++) возвращает старую временную копию (r-value).

Ключевые тезисы:

- l-value (locator value) — выражение, ссылающееся на конкретную область памяти с адресом (у него можно взять адрес через &).
- r-value — временное значение (литерал, результат арифметического выражения), не имеющее постоянного адреса в памяти.
- Префиксная форма ++i изменяет переменную и возвращает ссылку на неё (l-value), а постфиксная i++ создаёт копию старого значения (r-value) и увеличивает переменную.

Категории значений: L-value и R-value выражения

Исторически названия произошли от позиций относительно оператора присваивания left-value (слева от =) и right-value (справа от =), но в C++ они определяют фундаментальную категорию выражения:

- L-value (Locator value): выражение, которое идентифицирует некий существующий в памяти объект (переменную, элемент массива arr[i], разыменованный указатель *ptr). У любого l-value можно узнать адрес в памяти с помощью оператора &.
- Модифицируемое l-value: неконстантное l-value, которому можно присвоить новое значение (стоять слева от знака =). Константа const int c = 5; является немодифицируемым l-value.
- R-value (Read value / временное выражение): литералы (кроме строковых), результаты арифметических операций (a + b), возвраты функций по значению.

R-value живёт только до конца вычисления текущего полного выражения и не может стоять слева от =.

Операторы присваивания, инкремента (++) и декремента (--)

Простой оператор присваивания (=) и составные операторы (+=, -=, *=, /=, %=, <<=, >>=, &=, ^=, |=) обладают ПРАВОЙ ассоциативностью (выполняются справа налево) и возвращают в качестве результата само l-value левого операнда. Поэтому в C++ допустима цепочка: a = b = c = 0;

Форма оператора	Синтаксис	Возвращаемое значение	Категория результата в C++
Префиксный инкремент / декремент	++x / --x	Сначала изменяет x на 1, затем возвращает САМУ переменную x с новым значением	L-value (в C++ допустимо написать ++++x или &(++x))
Постфиксный инкремент / декремент	x++ / x--	Сохраняет старое значение во временную копию, изменяет x на 1, возвращает СТАРУЮ копию	R-value (написать (x++)++ или &(x++) — ошибка компиляции!)

Пример кода C++: Различие префиксного (++x) и постфиксного (x++) инкремента и l-value

Демонстрация возвращаемых значений префиксной и постфиксной форм, а также того, почему ++x является l-value.

```
#include <iostream>

int main() {
    int a = 5;
    int b = ++a; // Сначала a становится 6, затем b получает 6
    std::cout << "После b = ++a: a = " << a << ", b = " << b << "
";

    int c = 5;
    int d = c++; // Сначала d получает старое значение 5, затем c станов
    std::cout << "После d = c++: c = " << c << ", d = " << d << "
";

    // В C++ префиксный инкремент возвращает l-value (ссылку на саму пер
```

```
    int x = 10;
    ++x = 100; // Корректно! Сначала x станет 11, затем в x запишется 10
    std::cout << "После ++x = 100: x = " << x << "
";

    // Ассоциативность присваивания справа налево:
    int p, q;
    p = q = 42;
    std::cout << "Цепочка присваивания: p = " << p << ", q = " << q << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
После b = ++a: a = 6, b = 6
После d = c++: c = 6, d = 5
После ++x = 100: x = 100
Цепочка присваивания: p = 42, q = 42
```

Совет для экзамена: Никогда не модифицируйте одну и ту же переменную дважды в одном выражении без точки следования, например: i = i++ + ++i; — в стандарте C++ это классическое неопределённое поведение (Undefined Behavior, UB)!

БИЛЕТ №9 · УПРАВЛЕНИЕ И ФУНКЦИИ

9. Условный оператор. Оператор запятая. Инструкция-выражение. Инструкции выбора if и switch. Инструкции передачи управления.

Управление потоком выполнения в C++ осуществляется с помощью тернарного условного оператора ?:, оператора последовательности «запятая», инструкций ветвления if-else и множественного выбора switch, а также переходов break, continue, return и goto.

Ключевые тезисы:

- Тернарный оператор cond ? expr1 : expr2 является выражением (возвращает значение), тогда как if-else — это инструкция (statement).
- Оператор «запятая» (a, b) вычисляет левый операнд, отбрасывает его результат, затем вычисляет правый операнд и возвращает его значение (имеет самый низкий приоритет в C++).
- В инструкции switch выражение выбора может быть ТОЛЬКО целочисленного или перечислимого (enum) типа; без break происходит «проваливание» (fallthrough) в следующий case.

Условный (тернарный) оператор, оператор запятая и инструкция-выражение

Тернарный условный оператор — единственный оператор в C++, принимающий три операнда: условие ? выражение_если_истина : выражение_если_ложь. Поскольку он возвращает значение, его можно использовать внутри инициализации констант или вывода в поток: const int maxVal = (a > b) ? a : b;

Оператор «запятая» (,) вычисляет операнды строго слева направо и возвращает значение и тип последнего (правого) операнда. Имеет самый низкий приоритет среди всех операторов C++ (даже ниже присваивания =!). Чаще всего применяется в заголовках цикла for (int i = 0, j = n; i < j; ++i, --j).

Инструкция-выражение (Expression statement) — это любое выражение, завершённое точкой с запятой (;), например: x = 5; ++count; std::cout << x; Пустая инструкция состоит из одной точки с запятой ; и ничего не делает.

Инструкции выбора if-else и switch-case

Инструкция if (условие) проверяет произвольное логическое выражение. Если за if идут несколько вложенных if и один else, то else всегда относится к ближайшему предшествующему не закрытому if.

Инструкция множественного выбора switch (выражение) сопоставляет целочисленное (int, char, enum) выражение с константными метками case:

- Метки case должны быть константными выражениями времени компиляции (constexpr / литералы). Использование переменных или вещественных чисел (double) или строк (std::string) в case запрещено!
- Если в конце ветки case не поставить break;, выполнение беспрепятственно перейдёт («провалится» — fallthrough) к инструкциям следующего case.
- Ветка default: выполняется, если ни одна из меток case не совпала со значением выражения.

Инструкции безусловной передачи управления

Четыре ключевых оператора изменяют естественный порядок выполнения инструкций:

Инструкция	Где применяется	Действие
break;	Внутри циклов и внутри switch	Немедленно прерывает выполнение самого внутреннего цикла или блока switch.
continue;	ТОЛЬКО внутри циклов (for, while, do-while)	Досрочно завершает текущую итерацию цикла и переходит к следующей итерации.
return [expr];	Внутри любой функции	Прерывает выполнение всей функции и возвращает управление (и значение expr) в точку вызова.
goto метка;	В пределах одной функции	Безусловный прыжок на указанную метку label: (нарушает структурное программирование, применяется крайне

Инструкция	Где применяется	Действие
		редко для выхода из глубоко вложенных циклов).

Пример кода C++: Тернарный оператор, оператор запятая и switch с объединением меток case

Применение оператора запятая с учётом приоритета, тернарного оператора и конструкции switch.

```
#include <iostream>

int main() {
    int a = 10, b = 25;
    // Тернарный оператор возвращает максимум
    int maxVal = (a > b) ? a : b;

    // Оператор запятая: скобки обязательны, так как приоритет ',' ниже
    int x = (a += 5, b += 10, a + b); // a=15, b=35, x = 15 + 35 = 50
    std::cout << "maxVal = " << maxVal << ", x = " << x << "
";

    int day = 6;
    switch (day) {
        case 1: case 2: case 3: case 4: case 5:
            std::cout << "Рабочий будний день
";
            break; // Предотвращает проваливание в выходные
        case 6: case 7:
            std::cout << "Выходной день!
";
            break;
        default:
            std::cout << "Некорректный номер дня
";
            break;
    }
    return 0;
}
```

Результат выполнения (stdout):

```
maxVal = 25, x = 50
Выходной день!
```

Совет для экзамена: Запомните ловушку с оператором запятая без скобок: в записи int x = 10, 20; это ошибка синтаксиса объявления, а в записи x = 10, 20; из-за высокого приоритета = сначала выполнится (x = 10), а число 20 просто вычислится вхолостую! Чтобы в x попало 20, нужны скобки: x = (10, 20);.

БИЛЕТ №10 · УПРАВЛЕНИЕ И ФУНКЦИИ

10. Организация циклов в C++. Цикл foreach. Ключевое слово auto.

В C++ представлены три классических цикла (while с предусловием, do-while с постусловием, параметрический for) и современный цикл по коллекции (range-based for / foreach). Ключевое слово auto позволяет компилятору автоматически выводить тип переменной по её инициализатору.

Ключевые тезисы:

- Цикл while и for могут не выполниться ни разу, если условие изначально ложно; цикл do-while гарантированно выполняется хотя бы 1 раз.
- В цикле range-based for (auto& elem : container) использование ссылки & позволяет изменять элементы контейнера и избегать лишнего копирования.
- Ключевое слово auto (с C++11) выводит тип на этапе компиляции, поэтому переменная с auto обязана быть сразу инициализирована.

Классические циклы: while, do-while и for

Циклы позволяют многократно выполнять тело цикла, пока соблюдается условие продолжения:

- Цикл с предусловием while (условие) { тело }: перед каждой итерацией проверяется условие. Если оно с самого начала равно false, тело цикла не выполнится ни разу. Удобен, когда число итераций заранее неизвестно (чтение до конца файла, сходимость ряда с точностью eps).
- Цикл с постусловием do { тело } while (условие);: сначала выполняется тело цикла, и только потом проверяется условие. Гарантирует минимум одно выполнение (идеален для диалогового меню и запроса корректного ввода у пользователя). Обратите внимание на точку с запятой в конце!
- Цикл со счётчиком (параметрический) for (инициализация; условие; шаг) { тело }: объединяет подготовку счётчика, проверку условия перед итерацией и

модификацию счётчика после итерации. Любая из трёх секций заголовка может быть опущена; for (;;) создаёт бесконечный цикл.

Цикл foreach (Range-based for) и автоматический вывод типа auto

Начиная со стандарта C++11, ключевое слово auto указывает компилятору вывести тип переменной автоматически из типа присваиваемого выражения (на этапе компиляции, без потери статической типизации!).

Цикл по диапазону (range-based for, аналог foreach) имеет синтаксис: for (объявление_элемента : коллекция) { ... } и работает со статическими массивами, std::vector, std::array, std::string и любыми контейнерами с методами begin() и end():

Синтаксис заголовка foreach	Что происходит с элементами	Когда использовать
for (auto x : arr)	Каждый элемент КОПИРУЕТСЯ в локальную переменную x	Только для чтения простых примитивных типов (int, char, double). Изменение x НЕ меняет массив!
for (auto& x : arr)	x — ссылка на реальный элемент контейнера	Когда нужно МОДИФИЦИРОВАТЬ элементы массива прямо внутри цикла.
for (const auto& x : arr)	x — константная ссылка (без копирования и с защитой от изменения)	Только для чтения тяжёлых объектов (std::string, структуры, вложенные векторы).

Пример кода C++: Модификация элементов через for (auto& x : arr) и чтение через const auto&

Показывает, как удвоить все элементы массива с помощью ссылки auto& и вывести результат без лишнего копирования.

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> scores = {10, 20, 30, 40};
```

```
    // 1. Модификация элементов коллекции по ссылке (auto&)
    for (auto& val : scores) {
        val *= 2; // Изменяет сам элемент внутри вектора
    }

    // 2. Безопасное чтение элементов по константной ссылке (const auto&
    std::cout << "Удвоенные значения: ";
    for (const auto& val : scores) {
        std::cout << val << " ";
    }
    std::cout << "
";

    // 3. Цикл с постусловием do-while (выполняется минимум 1 раз)
    int attempts = 0;
    do {
        ++attempts;
    } while (attempts < 0); // Условие ложно, но 1 раз тело уже выполнил
    std::cout << "Число выполнений do-while: " << attempts << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
Удвоенные значения: 20 40 60 80
Число выполнений do-while: 1
```

Совет для экзамена: Помните: цикл range-based for (auto x : arr) НЕ работает с динамическими массивами, переданными через обычный указатель int* ptr, так как компилятор не знает размер массива по указателю! Он работает только со статическими массивами в области их объявления и контейнерами STL.

БИЛЕТ №11 · УПРАВЛЕНИЕ И ФУНКЦИИ

11. Подпрограммы. Определение и объявление подпрограмм. Процедуры и функции.

Подпрограмма — это именованный, логически самостоятельный блок кода, который можно многократно вызывать из разных частей программы. В C++ и процедуры, и функции синтаксически оформляются как функции (процедура — это функция с типом возвращаемого значения void).

Ключевые тезисы:

- Подпрограммы устраняют дублирование кода (принцип DRY — Don’t Repeat Yourself), упрощают тестирование и повышают читаемость.
- Объявление (прототип) сообщает компилятору имя функции, тип возврата и типы параметров; определение содержит само тело { ... }.
- Процедура выполняет действие без возврата значения (в C++ тип возврата void), а функция вычисляет и возвращает результат через return expr;.

Понятие и назначение подпрограмм

Подпрограмма — это выделенный и поименованный фрагмент алгоритма, решающий законченную частную задачу, к которому можно обращаться по имени из главной программы или других подпрограмм.

Главные преимущества использования подпрограмм: 1) сокращение размера исходного и объектного кода за счёт устранения дублирования, 2) декомпозиция сложной задачи на простые независимые шаги, 3) локализация ошибок (достаточно исправить баг в одной функции, а не в десяти местах программы), 4) возможность повторного использования кода в других проектах.

Процедуры и функции: реализация в языке C++

В классической теории алгоритмов (и в языках вроде Pascal или Ada) подпрограммы строго делятся на два вида:

- Функция (Function): подпрограмма, основная цель которой — вычислить и вернуть одно результирующее значение в точку вызова. Вызов функции является выражением и может использоваться внутри формул: y = sin(x) + sqr(a);.
- Процедура (Procedure): подпрограмма, которая выполняет последовательность действий (например, выводит таблицу на экран, сортирует массив, записывает лог в файл), но не возвращает результирующего значения. Вызов процедуры является отдельной инструкцией.
- В языке C++ нет отдельного ключевого слова procedure: любая подпрограмма синтаксически является функцией. Роль процедуры играет функция, у которой тип возвращаемого значения указан как void.

Различие между объявлением (прототипом) и определением

В C++ действует правило: любой идентификатор должен быть объявлен до его первого использования. Поэтому различают:

- Объявление функции (Declaration / Прототип): содержит только заголовок функции с точкой с запятой на конце (например, double calcArea(double radius);). Имена параметров в прототипе необязательны (double calcArea(double);). Память под код не выделяется — компилятор лишь запоминает сигнатуру для проверки типов при вызове.
- Определение функции (Definition): содержит заголовок и тело функции в фигурных скобках { ... }. Согласно правилу одного определения (ODR — One Definition Rule), функция может быть объявлена сколько угодно раз, но определена в программе ровно один раз.

Пример кода C++: Прототипы, функция (с возвратом значения) и процедура (void) в C++

Размещение прототипов до main(), вызов функции внутри выражения и вызов void-процедуры для форматированного вывода.

```
#include <iostream>

// 1. Объявления (прототипы) подпрограмм до функции main
int computeSquare(int x);              // Прототип функции
void printBanner(int input, int res);  // Прототип процедуры (void)

int main() {
    int number = 9;
```

```
    // Вызов функции как части выражения присваивания
    int result = computeSquare(number);
    // Вызов процедуры как отдельной инструкции
    printBanner(number, result);
    return 0;
}

// 2. Определения (реализации) подпрограмм после функции main
int computeSquare(int x) {
    return x * x;
}

void printBanner(int input, int res) {
    std::cout << "Квадрат числа " << input << " равен " << res << std::e
    // return; в конце void-функции необязателен
}
```

Результат выполнения (stdout):

```
Квадрат числа 9 равен 81
```

Совет для экзамена: Если нередкая ошибка «Unresolved external symbol» возникает при сборке, это значит, что вы написали объявление (прототип) функции и вызвали её, но забыли написать её определение (тело)! Компилятор поверил прототипу, а компоновщик (линкер) не нашёл машинного кода функции.

БИЛЕТ №12 · УПРАВЛЕНИЕ И ФУНКЦИИ

12. Объявление и определение функции в C++. Формальные и фактические параметры. Соответствие типов в формальных и фактических параметрах. Способы передачи параметров (по значению и по адресу). Параметры по умолчанию. Встроенные функции. Стандартные функции. Побочный эффект подпрограмм.

Функции в C++ принимают фактические аргументы, которые связываются с формальными параметрами по значению (копия), по указателю (адрес) или по ссылке (псевдоним). Поддерживаются параметры по умолчанию и встраиваемые (inline) функции.

Ключевые тезисы:

- Формальные параметры указываются в заголовке функции при её объявлении/определении, а фактические (аргументы) передаются в скобках в момент вызова.
- Передача по значению копирует аргумент (оригинал защищён); передача по адресу (указателю T* или ссылке T&) позволяет функции модифицировать внешнюю переменную.
- Параметры по умолчанию задаются только с правого края списка параметров и указываются в прототипе функции (не дублируются в определении).

Формальные и фактические параметры и способы их передачи

Формальные параметры — это локальные переменные функции, перечисленные в её заголовке. Фактические параметры (аргументы) — конкретные переменные,

константы или выражения, передаваемые при вызове функции. Количество, порядок и типы фактических параметров должны соответствовать формальным (либо допускать неявное приведение типов).

Способ передачи	Синтаксис параметра	Механизм работы в памяти	Влияние на исходный аргумент
По значению (By Value)	void f(int x)	В стеке создаётся локальная КОПИЯ аргумента.	Изменения параметра x внутри функции НЕ влияют на переменную в вызывающем коде.
По указателю (по адресу)	void f(int* p)	В функцию передаётся адрес переменной (&a). Доступ к данным через разыменование *p.	Позволяет изменять оригинальную переменную. Требует проверки p != nullptr.
По ссылке (по адресу в C++)	void f(int& r)	Параметр r становится прямым синонимом (псевдонимом) переданной переменной.	Изменение r напрямую меняет оригинал без синтаксиса звездочек. Для защиты от изменений используют const T&.

Параметры по умолчанию, inline-функции и побочные эффекты

Параметры по умолчанию (Default arguments): позволяют задать значение параметра, которое будет подставлено автоматически, если при вызове аргумент опущен. Правило: все параметры по умолчанию обязаны располагаться СПРАВА (в конце списка параметров) без пропусков: void log(const char* msg, int level = 1, bool newline = true);.

Встроенные функции (inline): ключевое слово inline рекомендует компилятору вместо обычного вызова функции (с сохранением регистров в стек и прыжком call/ret) подставить машинный код тела функции непосредственно в точку вызова. Это ускоряет выполнение маленьких часто вызываемых функций.

Стандартные функции: готовые функции из стандартной библиотеки C++ (<cmath>, <cstdlib>, <cctype>, <algorithm>), такие как std::sqrt, std::sin, std::abs, std::sort.

Побочный эффект подпрограммы (Side effect): любое действие функции, кроме возврата значения через return, которое изменяет внешнее состояние программы (модификация глобальной или статической переменной, изменение аргумента, переданного по указателю/ссылке, вывод на экран или запись в файл). Функция без побочных эффектов, зависящая только от своих аргументов, называется чистой (pure function).

Пример кода C++: Сравнение передачи по значению, по указателю, по ссылке и параметры по умолчанию

Показывает, как изменяется (или не изменяется) внешняя переменная при трёх способах передачи параметров, а также работу inline-функции с параметром по умолчанию.

```
#include <iostream>

void passByValue(int x)   { x += 100; } // Меняет только локальную копию
void passByPointer(int* p){ if (p) *p += 100; } // Меняет оригинал по ад
void passByRef(int& r)    { r += 100; } // Меняет оригинал через ссылку

// Встроенная функция с параметром по умолчанию (exp = 2)
inline int power(int base, int exp = 2) {
    int res = 1;
    for (int i = 0; i < exp; ++i) res *= base;
    return res;
}

int main() {
    int a = 10, b = 10, c = 10;
    passByValue(a);
    passByPointer(&b); // Передаём адрес переменной b
    passByRef(c);      // Передаём саму переменную c по ссылке

    std::cout << "По значению a = " << a << "
";
    std::cout << "По указателю b = " << b << "
";
    std::cout << "По ссылке c = " << c << "
";

    std::cout << "power(5) [по умолчанию квадрат] = " << power(5) << "
";
    std::cout << "power(2, 4) [явно 4-я степень] = " << power(2, 4) << "
";
```

```
    return 0;
}
```

Результат выполнения (stdout):

```
По значению a = 10
По указателю b = 110
По ссылке c = 110
power(5) [по умолчанию квадрат] = 25
power(2, 4) [явно 4-я степень] = 16
```

Совет для экзамена: Если у функции есть отдельно прототип (в .h файле) и отдельно определение (в .cpp файле), то значения параметров по умолчанию пишутся ТОЛЬКО ОДИН РАЗ — в прототипе! Повторное указание их в определении вызовет ошибку компиляции.

БИЛЕТ №13 · УПРАВЛЕНИЕ И ФУНКЦИИ

13. Рекурсия. Формы рекурсивных подпрограмм. Глубина и текущий уровень рекурсии. Зацикливание рекурсивных подпрограмм. Примеры неэффективности рекурсии.

Рекурсия — приём программирования, при котором функция вызывает саму себя напрямую или косвенно. Каждая корректная рекурсивная функция обязана иметь базовый случай (условие выхода) и рекурсивный шаг, приближающий аргументы к базовому случаю.

Ключевые тезисы:

- Формы рекурсии: прямая и косвенная (взаимная); линейная, хвостовая (tail recursion) и каскадная (древовидная).
- При каждом рекурсивном вызове в стеке выделяется новый стековый кадр (Stack Frame). Отсутствие базового случая приводит к переполнению стека (Stack Overflow).
- Классический пример неэффективности: наивное вычисление чисел Фибоначчи F(n) = F(n-1) + F(n-2) имеет экспоненциальную сложность O(2^n) из-за многократного пересчёта одних и тех же значений.

Понятие рекурсии, уровень и глубина рекурсии

Подпрограмма называется рекурсивной, если в процессе её выполнения происходит новый вызов этой же подпрограммы (до того, как завершился предыдущий вызов).

Любая рекурсивная функция состоит из двух частей: 1) Терминальная ветвь (базовый случай / граничное условие), в которой ответ возвращается сразу без рекурсивного вызова; 2) Рекурсивный шаг (декомпозиция), сводящий задачу размерности N к задаче меньшей размерности.

- Текущий уровень рекурсии: количество активных (ещё не завершившихся) вложенных вызовов рекурсивной функции в данный момент времени (исходный вызов из main часто считают уровнем 0 или 1).
- Глубина рекурсии: максимальный достигнутый уровень вложенности рекурсивных вызовов за всё время решения задачи (определяет пиковый расход памяти в стеке вызовов).
- Зацикливание рекурсии (бесконечная рекурсия): возникает, если базовый случай отсутствует или рекурсивный шаг не приближает аргумент к базовому случаю (например, fact(n) вызывает fact(n + 1)). В отличие от бесконечного цикла while, бесконечная рекурсия быстро исчерпывает память стека и аварийно роняет программу с ошибкой Stack Overflow.

Формы рекурсивных подпрограмм и примеры неэффективности

В зависимости от структуры вызовов выделяют следующие виды рекурсии:

- Почему рекурсия может быть неэффективной? Во-первых, каждый вызов требует накладных расходов на создание кадра стека (сохранение адреса возврата, параметров и локальных переменных). Во-вторых, при каскадной рекурсии без мемоизации (например, fib(n) = fib(n-1) + fib(n-2)) одни и те же подзадачи вычисляются миллионы раз заново (экспоненциальное время O(2^n) вместо O(n) в цикле).

Форма рекурсии	**Описание**	**Пример**
Прямая (Direct)	Функция A вызывает непосредственно саму себя A().	Факториал, быстрая сортировка Хоара (QuickSort).
Косвенная / Взаимная (Indirect)	Функция A вызывает функцию B, а функция B вызывает A.	Синтаксический разбор арифметических выражений (выражение ↔ слагаемое ↔ множитель).
Линейная	За один вызов функции порождается не более ОДНОГО рекурсивного вызова.	Факториал n! = n * (n-1)!, бинарный поиск.
Хвостовая (Tail recursion)	Частный случай линейной рекурсии, когда рекурсивный	Компилятор может оптимизировать её в

Форма рекурсии	Описание	Пример
	вызов является САМОЙ ПОСЛЕДНЕЙ операцией перед return.	обычный цикл без затрат памяти стека (Tail Call Optimization).
Каскадная / Древовидная	Тело функции содержит ДВА и более рекурсивных вызова.	Числа Фибоначчи, Ханойские башни, обход дерева.

Пример кода C++: Сравнение наивной каскадной рекурсии Фибоначчи и эффективной итеративной версии

Подсчёт количества вызовов функции при рекурсивном вычислении 10-го числа Фибоначчи (177 вызовов!) по сравнению с линейным циклом.

```
#include <iostream>

int callCounter = 0;

// Неэффективная каскадная рекурсия O(2^n)
long long fibRecursive(int n) {
    ++callCounter;
    if (n <= 1) return n; // Базовый случай (условие выхода)
    return fibRecursive(n - 1) + fibRecursive(n - 2); // Два рекурсивных
}

// Эффективная итеративная версия O(n) по времени и O(1) по памяти
long long fibIterative(int n) {
    if (n <= 1) return n;
    long long prev = 0, curr = 1;
    for (int i = 2; i <= n; ++i) {
        long long next = prev + curr;
        prev = curr;
        curr = next;
    }
    return curr;
}

int main() {
    int n = 10;
    long long resRec = fibRecursive(n);
    std::cout << "fibRecursive(" << n << ") = " << resRec
              << " (потребовалось вызовов: " << callCounter << ")
";
```

```
    std::cout << "fibIterative(" << n << ") = " << fibIterative(n)
              << " (за 1 проход цикла)
";
    return 0;
}
```

Результат выполнения (stdout):

```
fibRecursive(10) = 55 (потребовалось вызовов: 177)
fibIterative(10) = 55 (за 1 проход цикла)
```

Совет для экзамена: Для fibRecursive(40) потребуется уже более 331 миллиона рекурсивных вызовов, тогда как цикл выполнит всего 39 сложений! На экзамене это лучший пример неэффективности рекурсии.

БИЛЕТ №14 · УПРАВЛЕНИЕ И ФУНКЦИИ

14. Перегрузка функций. Ошибки, возникающие при перегрузке функций.

Перегрузка функций (Function Overloading) — механизм статического полиморфизма в C++, позволяющий объявлять в одной области видимости несколько функций с одинаковым именем, но различающимися списками формальных параметров (сигнатурой).

Ключевые тезисы:

- Компилятор выбирает нужную перегруженную функцию на этапе компиляции по количеству, типам или порядку следования параметров.
- Тип возвращаемого значения НЕ входит в сигнатуру для разрешения перегрузки! Две функции, отличающиеся только типом возврата, вызовут ошибку компиляции.
- Типичные ошибки: неоднозначность вызова (Ambiguous call) при неявном приведении типов и конфликт перегруженных функций с параметрами по умолчанию.

Правила перегрузки функций и искажение имён (Name Mangling)

В языке C все функции обязаны иметь уникальные имена. В C++ компилятор применяет механизм декорирования (искажения) имён — Name Mangling: на этапе компиляции к имени функции в объектном файле автоматически дописываются коды типов её параметров (например, print(int) превращается в _Z5printi, а print(double) — в _Z5printd).

Функции считаются корректно перегруженными, если их списки параметров отличаются хотя бы одним из признаков: 1) количеством параметров, 2) типами параметров, 3) порядком следования разнотипных параметров или наличием const у ссылочного/указательного параметра.

Типичные ошибки и неоднозначности при перегрузке

При неправильном проектировании перегруженных функций компилятор выдаёт ошибку неоднозначности (call of overloaded function is ambiguous) или повторного определения:

Вид ошибки при перегрузке	Некорректный код	Причина ошибки
Отличие только типом возврата	int calc(int x); double calc(int x);	Вызов calc(5); не позволяет компилятору понять, какую из двух функций вызывать.
Конфликт параметров по умолчанию	void f(int a); void f(int a, int b = 0);	При вызове с одним аргументом f(10); подходят ОБЕ функции одновременно.

Неоднозначное приведение типов	void g(float x);
	void g(long x);
	g(3.14); // 3.14 имеет тип double!
		Литерал 3.14 (double) с одинаковым приоритетом может быть сужен и до float, и до long.

Квалификатор const при передаче по значению	void h(int x); void h(const int x);	Для копии по значению int и const int — одно и то же объявление (повторное определение). Но const int& и int& различаются!

Пример кода C++: Корректная перегрузка функции вычисления площади и обход неоднозначности

Три перегруженные версии функции area() для квадрата, прямоугольника и круга.

```
#include <iostream>

// 1. Площадь квадрата по целочисленной стороне
int area(int side) {
    return side * side;
}

// 2. Площадь прямоугольника (отличается количеством параметров)
int area(int width, int height) {
    return width * height;
}

// 3. Площадь круга (отличается типом параметра double)
double area(double radius) {
    return 3.1415926535 * radius * radius;
```

```
}

int main() {
    std::cout << "Квадрат (5): " << area(5) << "
";
    std::cout << "Прямоугольник (4, 6): " << area(4, 6) << "
";
    std::cout << "Круг (2.5): " << area(2.5) << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
Квадрат (5): 25
Прямоугольник (4, 6): 24
Круг (2.5): 19.6349
```

Совет для экзамена: Запомните: typedef или using не создают нового типа, а лишь вводят синоним! Поэтому перегрузить void f(int) и void f(size_t) можно (если size_t — unsigned long long), а void f(int) и void f(MyInt), где typedef int MyInt; — нельзя.

БИЛЕТ №15 · УПРАВЛЕНИЕ И ФУНКЦИИ

15. Указатели на функции. Callback-функции.

Скомпилированный машинный код функции располагается в оперативной памяти (сегмент кода .text) и имеет начальный адрес. Указатель на функцию хранит этот адрес и позволяет динамически выбирать вызываемую подпрограмму и передавать одну функцию внутрь другой в качестве параметра (механизм обратного вызова — Callback).

Ключевые тезисы:

- Синтаксис объявления: ТипВозврата (*ИмяУказателя) (ТипыПараметров); — круглые скобки вокруг (*ИмяУказателя) обязательны!
- Имя функции без скобок вызова автоматически преобразуется в её адрес в памяти.
- Callback-функция — это функция, адрес которой передаётся в другую функцию (например, универсальную сортировку или обработчик событий) для вызова в нужный момент.

Объявление и использование указателей на функции

Как и любая переменная, функция после загрузки программы в ОЗУ размещается по определённому адресу в кодовом сегменте памяти. Указатель на функцию строго типизирован: он может указывать только на функции с точно такой же сигнатурой (типом возврата и типами параметров).

Обратите внимание на приоритет скобок: запись int (*fp)(int, int); объявляет указатель fp на функцию, принимающую два int и возвращающую int. Если забыть скобки вокруг *fp и написать int *fp(int, int);, то это будет прототип обычной функции fp, возвращающей указатель int*!

Механизм обратного вызова (Callback-функции)

Callback (функция обратного вызова) — это паттерн программирования, при котором в универсальный алгоритм передаётся указатель на пользовательскую

функцию, определяющую конкретное правило обработки.

- Настраиваемые алгоритмы сортировки и фильтрации: функция сортировки знает, КАК переставлять элементы, но спрашивает у переданного ей компаратора compare(a, b), в каком порядке их нужно расположить (по возрастанию, по убыванию, по модулю).
- Численные методы математики: функция численного интегрирования методТрапеций(a, b, f) принимает указатель double (*f)(double) и может интегрировать любую переданную функцию (sin, cos, пользовательский полином).
- Событийно-ориентированное программирование (GUI, таймеры, сетевые запросы): регистрация обработчика нажатия кнопки onButtonClick.

Пример кода C++: Указатель на функцию и Callback-компаратор для универсальной сортировки

Сортировка одного и того же массива сначала по возрастанию, а затем по убыванию путём передачи разных Callback-функций сравнения.

```
#include <iostream>

// Псевдоним типа для указателя на функцию сравнения (компаратор)
using CompareCallback = bool (*)(int, int);

// Callback 1: правило сортировки по возрастанию
bool ascending(int a, int b) { return a > b; }

// Callback 2: правило сортировки по убыванию
bool descending(int a, int b) { return a < b; }

// Универсальная сортировка, принимающая Callback-функцию
void bubbleSortWithCallback(int arr[], int n, CompareCallback shouldSwap
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - 1 - i; ++j) {
            if (shouldSwap(arr[j], arr[j + 1])) { // Вызов функции через
                int tmp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = tmp;
            }
        }
    }
}

void printArray(const int arr[], int n) {
    for (int i = 0; i < n; ++i) std::cout << arr[i] << " ";
```

```
    std::cout << "
";
}

int main() {
    int data[] = {42, 7, 19, 3, 25};
    int n = 5;

    bubbleSortWithCallback(data, n, ascending);
    std::cout << "По возрастанию: ";
    printArray(data, n);

    bubbleSortWithCallback(data, n, descending);
    std::cout << "По убыванию:    ";
    printArray(data, n);
    return 0;
}
```

Результат выполнения (stdout):

```
По возрастанию: 3 7 19 25 42
По убыванию:    42 25 19 7 3
```

Совет для экзамена: Для упрощения чтения сложных типов указателей на функции в C++11 и новее всегда используйте конструкцию using Action = int(*)(double, double); вместо громоздкого typedef int (*Action)(double, double);.

БИЛЕТ №16 · УПРАВЛЕНИЕ И ФУНКЦИИ

16. Макросы. Преимущества и недостатки использования макросов.

Макрос (макроопределение) задаётся директивой препроцессора #define и выполняет текстовую замену идентификатора или макроса с параметрами в исходном коде ещё ДО начала этапа компиляции.

Ключевые тезисы:

- Препроцессор не знает синтаксиса C++ и типов данных — он выполняет чисто механическую текстовую подстановку.
- В макросах-функциях (#define SQR(x) ((x) * (x))) каждый параметр и всё выражение целиком ОБЯЗАТЕЛЬНО нужно брать в круглые скобки.
- Главные опасности макросов: отсутствие контроля типов, изменение приоритета операций без скобок и многократное вычисление аргументов с побочными эффектами (например, SQR(++a)).

Понятие макросов: константы и макросы с параметрами

Директива #define ИМЯ ТЕКСТ_ЗАМЕНЫ указывает препроцессору заменить все последующие вхождения лексемы ИМЯ (кроме строковых литералов) на ТЕКСТ_ЗАМЕНЫ. Директива #undef ИМЯ отменяет действие ранее определённого макроса.

Существуют два типа макросов: 1) Макроконстанты без параметров (#define MAX_SIZE 1024), 2) Макросы с параметрами (макрофункции), например: #define MAX(a, b) (((a) > (b)) ? (a) : (b)). Между именем макроса и открывающей скобкой (НЕ ДОЛЖНО быть пробела!

Преимущества и недостатки макросов (почему в C++ предпочитают constexpr и inline)

В языке C макросы активно применялись для создания констант и быстрых подстановок, но в C++ большинство их задач безопаснее решают константы constexpr, встраиваемые функции inline и шаблоны template:

Преимущества макросов	Недостатки и ловушки макросов
Нет накладных расходов на вызов функции во время работы программы (код вставляется напрямую).	Полное отсутствие проверки типов данных (макрос может сгенерировать бессмысленный код со сложными ошибками).
Доступ к специальным предопределённым макросам для логирования: __FILE__, __LINE__, __func__, __DATE__.	Нарушение порядка операций при отсутствии скобок: если #define SQR(x) x*x, то SQR(2 + 3) развернётся в 2 + 3 * 2 + 3 = 11 вместо 25!
Используются в директивах условной компиляции (#ifdef) и стражах включения (#ifndef).	Двойное вычисление аргументов с побочным эффектом: даже со скобками SQR(++a) превратится в ((++a) * (++a)), увеличив a дважды и вызвав UB!
Операторы препроцессора # (превращение аргумента в строку) и ## (склейка лексем) недоступны обычным функциям.	Макросы не видны отладчику (нельзя поставить точку останова внутри многострочного макроса) и не подчиняются пространствам имён (namespace).

Пример кода C++: Классические ловушки макроса SQR(x) и безопасная альтернатива в C++

Наглядная демонстрация ошибки приоритета операций без скобок в макросе и сравнение с шаблонной inline-функцией.

```
#include <iostream>

// ОПАСНЫЙ макрос без защитных скобок
#define BAD_SQR(x) x * x

// ПРАВИЛЬНЫЙ макрос со скобками вокруг каждого аргумента и всего выраже
#define SAFE_PAREN_SQR(x) ((x) * (x))

// ИДЕАЛЬНОЕ решение в C++: шаблонная inline/constexpr функция
template <typename T>
constexpr T cppSqr(T x) {
    return x * x;
}

int main() {
    // 1. Ловушка приоритета операций: 2 + 3 * 2 + 3 = 11 (вместо 25!)
```

```
    std::cout << "BAD_SQR(2 + 3) = " << BAD_SQR(2 + 3) << " (ошибка: 2 +
";
    std::cout << "SAFE_PAREN_SQR(2 + 3) = " << SAFE_PAREN_SQR(2 + 3) <<
";

    // 2. Безопасный вызов функции с инкрементом (аргумент вычисляется р
    int val = 4;
    std::cout << "cppSqr(++val) = " << cppSqr(++val) << ", val = " << va
";

    // 3. Полезное применение макросов: отладочная информация о файле и
    std::cout << "Лог из функции: " << __func__ << ", строка: " << __LIN
";
    return 0;
}
```

Результат выполнения (stdout):

```
BAD_SQR(2 + 3) = 11 (ошибка: 2 + 3*2 + 3)
SAFE_PAREN_SQR(2 + 3) = 25
cppSqr(++val) = 25, val = 5
Лог из функции: main, строка: 25
```

Совет для экзамена: На экзамене почти всегда просят показать два примера, где макрос #define SQR(x) x*x ломается: 1) при передаче суммы SQR(a + b) -> a + b * a + b; 2) при делении на макрос 100 / SQR(2) -> 100 / 2 * 2 = 100 (вместо 25), если забыты внешние скобки вокруг всего макроса!

17. Шаблоны функций.

Шаблон функции (Function Template) — инструмент обобщённого программирования в C++, позволяющий описать единый алгоритм, параметризованный одним или несколькими типами данных. Компилятор автоматически генерирует конкретные версии функции для используемых типов.

Ключевые тезисы:

- Синтаксис: template <typename T> (или template <class T>) перед заголовком функции.
- Процесс генерации компилятором конкретной функции из шаблона для заданного типа называется инстанцированием шаблона (Template Instantiation).
- Поддерживается автоматический вывод типов параметров (Type Deduction), явное указание типа func<double>(a, b) и явная специализация шаблона template <> для особых типов.

Понятие шаблона функции и механизм инстанцирования

Часто алгоритм работы функции (например, обмен двух переменных swap, поиск максимума max, сортировка массива) абсолютно одинаков для int, double, std::string и пользовательских структур. Вместо ручного копирования кода и написания десятка перегруженных функций в C++ создаётся шаблон функции.

Сам по себе шаблон функции не является готовым машинным кодом и не занимает места в сегменте кода, пока к нему не обратятся. Когда компилятор встречает вызов maxValue(10, 20), он подставляет тип int вместо параметра T и компилирует конкретную экземпляр-функцию int maxValue(int, int). Этот процесс называется инстанцированием шаблона.

Параметры шаблонов, вывод типов и специализация

В списке параметров шаблона template <...> ключевые слова typename и class полностью эквивалентны. Шаблон может принимать:

- Параметры-типы: template <typename T, typename U> — позволяют передавать аргументы как одного типа, так и разных типов.
- Нетиповые параметры (Non-type template parameters): константы времени компиляции, например целочисленный размер массива: template <typename T, size_t N>.
- Разрешение неоднозначности при выводе типов: если шаблон объявлен как template <typename T> T add(T a, T b), то вызов add(5, 3.14) вызовет ошибку компиляции, так как первый аргумент int, а второй double. Решение: явно указать тип при вызове add<double>(5, 3.14).
- Явная специализация шаблона (template <>): если для какого-то специфического типа (например, C-строк const char*, которые нельзя сравнивать оператором > по содержимому, так как сравнятся адреса) общий алгоритм не подходит, программист может написать отдельную специализированную реализацию.

Пример кода C++: Обобщённый шаблон поиска максимума и его явная специализация для C-строк

Показывает работу шаблона getMax<T> для чисел и объектов std::string, а также явную специализацию через std::strcmp для указателей const char*.

```
#include <iostream>
#include <cstring>
#include <string>

// Основной обобщённый шаблон функции
template <typename T>
T getMax(T a, T b) {
    return (a > b) ? a : b;
}

// Явная специализация шаблона для C-строк (const char*),
// чтобы сравнивать строки лексикографически, а не адреса указателей!
template <>
const char* getMax<const char*>(const char* a, const char* b) {
    return (std::strcmp(a, b) > 0) ? a : b;
}

int main() {
    // Автоматический вывод типов T = int и T = double
```

```
    std::cout << "getMax(15, 42) = " << getMax(15, 42) << "
";
    std::cout << "getMax(3.14, 2.71) = " << getMax(3.14, 2.71) << "
";

    // Явное указание типа T = double при смешанных аргументах (int и do
    std::cout << "getMax<double>(5, 9.8) = " << getMax<double>(5, 9.8) <
";

    // Вызов специализированной версии для const char*
    const char* s1 = "Alpha";
    const char* s2 = "Omega";
    std::cout << "getMax C-strings: " << getMax(s1, s2) << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
getMax(15, 42) = 42
getMax(3.14, 2.71) = 3.14
getMax<double>(5, 9.8) = 9.8
getMax C-strings: Omega
```

Совет для экзамена: Важное правило для многофайловых проектов: и объявление, и полное определение (тело) шаблона функции должны находиться в заголовочном файле (.h / .hpp), так как компилятору нужен исходный текст шаблона в каждой единице трансляции для его инстанцирования под конкретные типы!

БИЛЕТ №18 · УПРАВЛЕНИЕ И ФУНКЦИИ

18. Директивы условной компиляции. Многофайловые проекты. Страж включения.

Многофайловые проекты разделяют интерфейс (заголовочные файлы .h) и реализацию (единицы трансляции .cpp). Директивы условной компиляции (#if, #ifdef, #ifndef, #endif) и стражи включения (Include Guards / #pragma once) управляют составом компилируемого кода и защищают от повторного включения заголовков.

Ключевые тезисы:

- В заголовочные файлы (.h) помещают объявления (прототипы функций, определения структур, классы, inline/template функции), а в файлы реализации (.cpp) — тела функций.
- Директива #include <file> ищет заголовок в системных каталогах стандартной библиотеки, а #include "file.h" — сначала в текущем каталоге проекта.
- Страж включения (#ifndef HEADER_H / #define HEADER_H / ... / #endif или #pragma once) предотвращает ошибку повторного определения типов при ромбовидном включении заголовков.

Директивы условной компиляции препроцессора

Условная компиляция позволяет включать или полностью исключать участки исходного кода до передачи его компилятору в зависимости от заданных макросов или целевой операционной системы:

- #ifdef МАКРОС (или #if defined(МАКРОС)): компилировать блок кода только если макрос был определён через #define или ключ компилятора -D.
- #ifndef МАКРОС (или #if !defined(МАКРОС)): компилировать блок кода только если макрос НЕ определён.
- #if КОНСТАНТНОЕ_ВЫРАЖЕНИЕ / #elif ... / #else / #endif: ветвление препроцессора по целочисленным константам (например, #if DEBUG_LEVEL >= 2).

- Практическое применение: кроссплатформенная разработка (#ifdef _WIN32 ... #elif defined(__linux__)), включение отладочных логов только в Debug-сборке и временное отключение больших блоков кода (#if 0 ... #endif).

Архитектура многофайловых проектов и Страж включения (Include Guard)

Крупные программы разбиваются на независимые модули. Каждый модуль обычно состоит из пары файлов: заголовочного файла интерфейса (math_utils.h) и файла реализации (math_utils.cpp). Каждый .cpp файл компилируется отдельно в объектный файл (.o / .obj) и называется единицей трансляции.

Проблема: если файл geometry.h подключает point.h, а main.cpp подключает и point.h, и geometry.h, то содержимое point.h скопируется в main.cpp ДВАЖДЫ. Это нарушит правило одного определения (ODR) для структур и классов и вызовет ошибку компиляции redefinition of struct Point.

Решение — Страж включения (Include Guard): обёртка всего содержимого .h-файла в проверку уникального макроса #ifndef / #define / #endif (либо современная директива #pragma once). При втором чтении файла в той же единице трансляции макрос уже определён, и препроцессор мгновенно пропускает дублирующийся текст.

Пример кода C++: Структура многофайлового проекта со стражем включения (Include Guard) и условной компиляцией

Показывает классический страж включения #ifndef в заголовочном файле и использование условной компиляции для отладочной печати.

```
// === Файл: vec2d.h (Заголовочный файл с Include Guard) ===
#ifndef VEC2D_H_INCLUDED
#define VEC2D_H_INCLUDED

struct Vec2D {
    double x;
    double y;
};

// Прототип функции (реализация в .cpp)
double dotProduct(const Vec2D& a, const Vec2D& b);

#endif // VEC2D_H_INCLUDED
```

```
// === Файл: main.cpp (Подключение и условная компиляция) ===
#include <iostream>
// Включаем режим отладочной трассировки
#define ENABLE_DEBUG_TRACE 1

double dotProduct(const Vec2D& a, const Vec2D& b) {
#if ENABLE_DEBUG_TRACE
    std::cout << "[DEBUG] Вычисление скалярного произведения...
";
#endif
    return a.x * b.x + a.y * b.y;
}

int main() {
    Vec2D u{3.0, 4.0}, v{2.0, 5.0};
    std::cout << "Результат (u, v) = " << dotProduct(u, v) << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
[DEBUG] Вычисление скалярного произведения...
Результат (u, v) = 26
```

Совет для экзамена: Помните: страж включения (#ifndef / #pragma once) защищает от повторного включения заголовка только ВНУТРИ ОДНОЙ единицы трансляции (.cpp файла). Если вы напишете обычную (не inline) функцию с телом прямо в .h-файле и подключите его в два разных .cpp файла, страж включения не спасёт от ошибки компоновщика (Multiple definition of function)!

19. Массивы. Статические массивы.

Массив — это упорядоченный набор однотипных элементов, расположенных в оперативной памяти непрерывно (подряд друг за другом) и доступных по целочисленному индексу за константное время O(1).

Ключевые тезисы:

- Размер статического массива обязан быть константой времени компиляции (constexpr / литерал) и не может меняться во время работы программы.
- Индексация в C++ всегда начинается с 0 и заканчивается N - 1. При частичной инициализации int a[5] = {1, 2}; оставшиеся элементы автоматически заполняются нулями.
- C++ НЕ контролирует выход индекса за границы статического массива: обращение к arr[N] или arr[-1] приводит к неопределённому поведению (Undefined Behavior).

Устройство статического массива в памяти и способы инициализации

Объявление статического одномерного массива имеет вид: Тип ИмяМассива[Размер]; (например, int grades[10];). Все элементы размещаются в едином непрерывном блоке памяти размером N * sizeof(Тип) байт.

Благодаря непрерывному размещению адрес любого i-го элемента вычисляется процессором мгновенно за O(1) по формуле: Адрес(a[i]) = Адрес(a[0]) + i * sizeof(Тип).

- Без инициализации внутри функции (int a[5];): элементы содержат случайный «мусор», оставшийся в стеке.
- Полная инициализация списком: int a[4] = {10, 20, 30, 40}; или с автоопределением размера компилятором: int a[] = {10, 20, 30, 40}; (размер будет равен 4).

- Частичная инициализация и обнуление: если в фигурных скобках указано меньше элементов, чем размер массива, все остальные элементы гарантированно обнуляются! Запись int a[100] = {}; или int a[100] = {0}; обнуляет весь массив целиком.
- Определение числа элементов статического массива: N = sizeof(arr) / sizeof(arr[0]) или (начиная с C++17) через стандартную функцию std::size(arr) из <iterator>.

Ограничения статических массивов C-стиля

Статические массивы унаследованы из языка C и обладают рядом низкоуровневых особенностей:

- Массивы нельзя присваивать друг другу целиком (a = b; — ошибка компиляции) и нельзя сравнивать оператором == по содержимому (a == b лишь сравнит адреса начала массивов!). Для копирования используют цикл или std::copy.
- Отсутствует встроенная проверка границ: запись в ячейку за пределами массива портит соседние локальные переменные на стеке.

Пример кода C++: Инициализация, вычисление размера и поиск минимума/максимума в статическом массиве

Показывает частичную инициализацию, расчёт количества элементов через sizeof и однопроходный алгоритм анализа массива.

```
#include <iostream>

int main() {
    // Первые 4 элемента заданы явно, остальные 2 автоматически станут 0
    int temps[6] = {18, 24, 15, 29};

    // Вычисление количества элементов статического массива
    int n = sizeof(temps) / sizeof(temps[0]); // 24 байта / 4 байта = 6

    int minVal = temps[0];
    int maxVal = temps[0];
    double sum = 0;

    std::cout << "Элементы массива (" << n << " шт.): ";
    for (int i = 0; i < n; ++i) {
        std::cout << temps[i] << " ";
        if (temps[i] < minVal) minVal = temps[i];
```

```
        if (temps[i] > maxVal) maxVal = temps[i];
        sum += temps[i];
    }
    std::cout << "
Минимум: " << minVal
              << ", Максимум: " << maxVal
              << ", Среднее: " << (sum / n) << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
Элементы массива (6 шт.): 18 24 15 29 0 0
Минимум: 0, Максимум: 29, Среднее: 14.3333
```

Совет для экзамена: Частая ловушка на экзамене: что сделает код int arr[10] = {5};? Многие думают, что все 10 элементов станут равны 5. На самом деле ТОЛЬКО arr[0] станет равен 5, а элементы с arr[1] по arr[9] будут инициализированы НУЛЯМИ (0)!

20. Введение в STL, контейнеры vector и array.

STL (Standard Template Library) — стандартная библиотека шаблонов C++, базирующаяся на контейнерах, итераторах и алгоритмах. Контейнер std::array — безопасная обёртка над статическим массивом фиксированного размера, а std::vector — динамический массив с автоматическим управлением памятью в куче.

Ключевые тезисы:

- Три главных компонента STL: Контейнеры (хранят данные), Итераторы (обеспечивают доступ к элементам) и Алгоритмы (сортировка, поиск, трансформация).
- Оператор [i] у std::vector и std::array работает без проверки границ (максимально быстро), а метод .at(i) проверяет индекс и выбрасывает исключение std::out_of_range при выходе за границы.
- У std::vector важно различать size() (текущее число элементов) и capacity() (сколько элементов поместится в выделенный буфер до реаллокации).

Архитектура стандартной библиотеки шаблонов (STL)

Библиотека STL (разработана Александром Степановым и включена в стандарт C++) разделяет структуры хранения данных и алгоритмы их обработки за счёт промежуточного звена — итераторов:

- 1. Контейнеры (Containers): шаблонные классы для хранения коллекций объектов. Делятся на последовательные (std::array, std::vector, std::deque, std::list), ассоциативные упорядоченные (std::set, std::map) и неупорядоченные хеш-таблицы (std::unordered_map).
- 2. Итераторы (Iterators): объекты, ведущие себя как обобщённые указатели. Они позволяют перебирать элементы любого контейнера единым образом (begin(), end(), ++it, *it).
- 3. Алгоритмы (Algorithms, заголовок <algorithm>): шаблонные функции (std::sort, std::find, std::reverse, std::count_if), которые работают с диапазонами

итераторов [first, last) и не зависят от типа контейнера.

Сравнение контейнеров std::array и std::vector

Оба контейнера гарантируют непрерывное расположение элементов в памяти и доступ по индексу за O(1), но принципиально отличаются способом выделения памяти:

- Как работает std::vector::push_back(x)? Если текущий size() меньше зарезервированной ёмкости capacity(), элемент добавляется в конец мгновенно за O(1). Если буфер заполнен (size == capacity), вектор автоматически выделяет в куче новый блок памяти (обычно в 1.5 или 2 раза больше), копирует/перемещает туда старые элементы и освобождает старый буфер (амортизированная сложность O(1)). Метод reserve(N) позволяет заранее выделить память и избежать лишних реаллокаций.

Характеристика	**std::array<T, N> (<array>)**	**std::vector<T> (<vector>)**
Размещение в памяти	В стеке (или внутри объекта), как обычный статический массив	Сам объект (3 указателя) в стеке, а буфер элементов — в динамической памяти (Куче / Heap)
Изменение размера	Фиксированный размер N на этапе компиляции	Динамический: растёт и сжимается во время работы (push_back, pop_back, resize, clear)
Копирование и сравнение	Поддерживает присваивание a = b и сравнение a == b целиком	Поддерживает глубокое копирование v1 = v2 и сравнение v1 == v2; автоматически освобождает память в деструкторе
Ключевые методы	size(), empty(), front(), back(), at(i), data(), fill(val)	push_back(x), emplace_back(x), pop_back(), size(), capacity(), reserve(n), resize(n), at(i), clear()

Пример кода C++: Работа с std::array и динамическим std::vector (size, capacity, at)

Демонстрация методов std::array, динамического добавления элементов в std::vector и безопасного доступа через .at().

```
#include <iostream>
#include <array>
#include <vector>

int main() {
    // 1. Статический контейнер std::array фиксированного размера 4
    std::array<int, 4> fixedArr = {10, 20, 30, 40};
    std::cout << "std::array size = " << fixedArr.size()
              << ", первый = " << fixedArr.front()
              << ", последний = " << fixedArr.back() << "
";

    // 2. Динамический контейнер std::vector
    std::vector<int> vec;
    vec.reserve(4); // Заранее резервируем память под 4 элемента без реа
    vec.push_back(100);
    vec.push_back(200);
    vec.push_back(300);

    std::cout << "std::vector size = " << vec.size()
              << ", capacity = " << vec.capacity() << "
";

    // Безопасное обращение через метод .at() с проверкой границ
    std::cout << "Элемент vec.at(1) = " << vec.at(1) << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
std::array size = 4, первый = 10, последний = 40
std::vector size = 3, capacity = 4
Элемент vec.at(1) = 200
```

Совет для экзамена: В отличие от сырых С-массивов, std::array и std::vector знают свой размер (.size()), не распадаются неявно в указатель при передаче в функцию и поддерживают прямое копирование через оператор присваивания =.

21. Указатели. Типизированные и безтиповые указатели. Операция разыменования и операция получения адреса. Арифметические операции над указателями. Проблемы и типичные ошибки при работе с указателями. Константные указатели и указатели на константу.

Указатель — переменная, значением которой является адрес другой ячейки оперативной памяти. C++ различает типизированные указатели (T*) и нетипизированные (void*), поддерживает адресную арифметику с масштабированием на sizeof(T) и тонкую настройку константности через позицию ключевого слова const.

Ключевые тезисы:

- Оператор & возвращает адрес переменной в памяти, а унарный оператор * (разыменование) предоставляет доступ к значению, лежащему по этому адресу.
- Нетипизированный указатель void* может хранить адрес любого объекта, но его НЕЛЬЗЯ разыменовывать (*p) или сдвигать (p + 1) без предварительного приведения типа.
- Чтение справа налево (или относительно *): const int* p — указатель на константу (нельзя менять *p); int* const p — константный указатель (нельзя менять сам адрес p).

Типизированные и безтиповые указатели, операции & и *

Любой указатель на одной и той же архитектуре имеет одинаковый размер в памяти (8 байт на 64-битной ОС или 4 байта на 32-битной ОС), так как хранит просто номер начального байта в виртуальном адресном пространстве.

Зачем тогда указателю нужен тип (int*, double*)? По двум причинам: 1) при разыменовании *ptr компилятор должен знать, сколько байт прочитать из памяти (4 байта для int или 8 байт для double) и как их интерпретировать; 2) при адресной арифметике ptr + 1 указатель сдвигается ровно на sizeof(Тип) байт вперёд.

Безтиповый указатель void* — универсальный указатель «на область памяти неизвестного типа». В него можно неявно записать адрес любого неконстантного типа, но перед разыменованием его обязаны явно привести к конкретному типу через static_cast<int*>(voidPtr).

Арифметика указателей и типичные ошибки при работе с указателями

Над типизированными указателями разрешены: 1) прибавление/вычитание целого числа (p + k, p - k, ++p, --p) — смещает адрес на k * sizeof(T) байт; 2) вычитание двух указателей одного типа (p2 - p1) — возвращает количество элементов типа T между ними (тип ptrdiff_t); 3) сравнение указателей (<, >, ==, !=). Складывать два указателя (p1 + p2) ЗАПРЕЩЕНО!

- 1. Неинициализированный («дикий» / Wild) указатель: int* p; *p = 5; — указывает в случайный адрес памяти, приводит к краху (Segmentation Fault). Всегда инициализируйте указатели значением nullptr!
- 2. Разыменование нулевого указателя (Null Pointer Dereference): int* p = nullptr; *p = 10; — аварийное завершение программы.
- 3. Висячий указатель (Dangling Pointer): указатель, который продолжает ссылаться на уже освобождённую память (после delete p; или при возврате из функции адреса её локальной переменной).
- 4. Утечка памяти (Memory Leak): потеря адреса динамически выделенной памяти до вызова delete (например, p = new int(5); p = nullptr;).

Константные указатели и указатели на константу

Положение ключевого слова const относительно звёздочки * кардинально меняет смысл ограничения:

Объявление	Название	Можно ли менять адрес (p = &b)?	Можно ли менять значение (*p = 99)?
const int* p (или int const* p)	Указатель на константу (const стоит ДО *)	ДА (указатель можно перенаправить)	НЕТ (данные по адресу доступны только для чтения)
int* const p = &a;	Константный указатель (const стоит ПОСЛЕ *)	НЕТ (привязан к одному адресу навсегда)	ДА (значение самой переменной менять можно)
const int* const p = &a;	Константный указатель на константу	НЕТ	НЕТ

Пример кода C++: Разыменование, адресная арифметика и различие const int* и int* const

Показывает работу операторов & и *, вычитание указателей и правила константности.

```
#include <iostream>

int main() {
    int x = 10, y = 20;

    // 1. Указатель на константу (const ДО *): адрес менять можно, значе
    const int* pToConst = &x;
    pToConst = &y; // Разрешено! Теперь смотрит на y
    // *pToConst = 50; // Ошибка компиляции!

    // 2. Константный указатель (const ПОСЛЕ *): значение *constPtr меня
    int* const constPtr = &x;
    *constPtr = 77; // Разрешено! Меняет переменную x на 77
    // constPtr = &y; // Ошибка компиляции!

    // 3. Арифметика указателей внутри массива
    int arr[] = {100, 200, 300, 400};
    int* p1 = &arr[1]; // Указывает на 200
    int* p2 = &arr[3]; // Указывает на 400
    std::cout << "Изменённый x = " << x << ", *pToConst = " << *pToConst
";
```

```
    std::cout << "Разность указателей (p2 - p1) = " << (p2 - p1) << " эл
";
    return 0;
}
```

Результат выполнения (stdout):

```
Изменённый x = 77, *pToConst = 200
Разность указателей (p2 - p1) = 2 элемента
```

Совет для экзамена: Мнемоническое правило для экзамена: проведите мысленную вертикальную черту через звёздочку *. Если const слева от * (const int * p) — константны данные (*p). Если const справа от * (int * const p) — константен сам указатель (p).

БИЛЕТ №22 · МАССИВЫ, STL И УКАЗАТЕЛИ

22. Массивы. Реализация одномерных массивов с помощью указателей.

В C++ имя одномерного массива в большинстве выражений автоматически преобразуется (распадается — array-to-pointer decay) в указатель на его первый (нулевой) элемент. Операция индексации arr[i] по стандарту языка тождественна разыменованию со сдвигом *(arr + i).

Ключевые тезисы:

- Имя массива arr эквивалентно адресу его нулевого элемента &arr[0].
- Обращение по индексу arr[i] компилятор всегда переводит в адресное выражение *(arr + i).
- Имя статического массива НЕ является изменяемым указателем (ему нельзя присвоить другой адрес arr = ptr или сделать ++arr), но обычный указатель int* p = arr; можно свободно сдвигать по массиву.

Связь имени массива и указателя (Array-to-Pointer Decay)

В языках C и C++ работа с одномерными массивами неразрывно связана с адресной арифметикой указателей. Имя статического массива arr в любом выражении (кроме операторов sizeof(arr) и &arr) неявно преобразуется в указатель типа T* на нулевой элемент массива (&arr[0]).

Поэтому для любого одномерного массива справедливы следующие тождества:

Запись через индексы	Запись через указатели	Смысл выражения
&arr[0]	arr	Адрес начального (нулевого) элемента массива
arr[0]	*arr	Значение нулевого элемента массива
&arr[i]	arr + i	Адрес i-го элемента массива (смещение на i * sizeof(T) байт)

Запись через индексы	Запись через указатели	Смысл выражения
arr[i]	*(arr + i)	Значение i-го элемента массива (разыменование сдвинутого адреса)

Отличия имени статического массива от переменной-указателя

Несмотря на взаимозаменяемость синтаксиса arr[i] и *(arr + i), имя статического массива и отдельная переменная-указатель int* p = arr; имеют важные различия:

- 1. Модифицируемость адреса: переменная-указатель p хранит адрес в собственной ячейке памяти, поэтому её можно сдвигать (++p, p += 2) или перенаправлять на другой массив. Имя массива arr жёстко привязано компилятором к началу массива в стеке и не является l-value (написать ++arr; или arr = p; — ошибка компиляции!).
- 2. Результат оператора sizeof: sizeof(arr) возвращает суммарный размер ВСЕХ элементов массива в байтах (например, 10 * 4 = 40 байт), тогда как sizeof(p) возвращает размер самого указателя (всегда 8 байт на 64-битной платформе).
- 3. Итерация по массиву через два указателя: классический идиоматический приём C++ — перебор массива указателем p от начала arr до указателя за последний элемент end = arr + n.

Пример кода C++: Доступ к элементам одномерного массива и обход без индексной переменной

Показывает эквивалентность arr[i] и *(arr + i), а также быстрый обход массива с помощью перемещающегося указателя.

```
#include <iostream>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    // 1. Эквивалентность индексной и указательной формы
    std::cout << "arr[2] = " << arr[2] << ", *(arr + 2) = " << *(arr + 2
";

    // 2. Обход массива с помощью бегущего указателя от начала до конца
    int* beginPtr = arr;          // Указывает на arr[0]
    int* endPtr = arr + 5;        // Указывает на ячейку сразу ЗА послед
```

```
    std::cout << "Удваиваем элементы через указатель: ";
    for (int* p = beginPtr; p < endPtr; ++p) {
        *p *= 2;                  // Модификация элемента через разыменс
        std::cout << *p << " ";
    }
    std::cout << "
";

    // 3. Различие sizeof для массива и для указателя
    std::cout << "sizeof(arr) = " << sizeof(arr)
              << " байт, sizeof(beginPtr) = " << sizeof(beginPtr) << " б
";
    return 0;
}
```

Результат выполнения (stdout):

```
arr[2] = 30, *(arr + 2) = 30
Удваиваем элементы через указатель: 20 40 60 80 100
sizeof(arr) = 20 байт, sizeof(beginPtr) = 8 байт
```

Совет для экзамена: Забавный факт для экзамена, доказывающий, что a[i] — это просто синтаксический сахар для *(a + i): так как сложение коммутативно (a + i == i + a), в C++ выражение 2[arr] абсолютно легально, компилируется и возвращает тот же элемент arr[2]!

23. Многомерные массивы. Реализация многомерных массивов с помощью указателей. Динамическое выделение памяти под одномерные и двумерные массивы.

Статический двумерный массив int m[R][C] хранится в памяти построчно единым блоком, а доступ m[i][j] раскрывается как *(*(m + i) + j). Динамические массивы создаются в куче оператором new[] и требуют обязательного освобождения через delete[] в обратном порядке.

Ключевые тезисы:

- В статической матрице m[R][C] элементы расположены в памяти последовательно по строкам (Row-major order), а m[i][j] эквивалентно *(*(m + i) + j).
- Динамический одномерный массив выделяется через int* a = new int[n]; и освобождается строго парным оператором delete[] a;.
- Динамический двумерный массив через массив указателей (int** mat = new int*[rows]) выделяется в 2 этапа (массив строк + каждая строка в цикле) и удаляется в обратном порядке!

Статические многомерные массивы и их указательная интерпретация

Двумерный массив (матрица) в C++ с объявлением int mat[ROWS][COLS]; логически представляет собой «одномерный массив из ROWS элементов, каждый из которых сам является одномерным массивом из COLS чисел int».

Физически в оперативной памяти все ROWS * COLS элементов статической матрицы лежат непрерывно друг за другом построчно (сначала вся 0-я строка, за ней вся 1-я строка и т.д.).

При обращении через указатели: выражение mat имеет тип int (*)[COLS] (указатель на строку из COLS элементов). Сдвиг (mat + i) перешагивает сразу через i целых

строк, *(mat + i) даёт адрес начала i-й строки (тип int*), а *(*(mat + i) + j) обращается к j-му элементу в i-й строке, что в точности равно mat[i][j].

Динамическое выделение памяти под одномерные и двумерные массивы

Если размер массива становится известен только во время работы программы (например, вводится пользователем или читается из файла) либо массив слишком велик для стека, память выделяется в Куче (Heap) с помощью оператора new[]:

- Одномерный динамический массив: выделение int* arr = new int[n](); (пустые скобки в конце обнуляют массив). Освобождение: delete[] arr; arr = nullptr;. Использование простого delete arr; без квадратных скобок для массива приводит к неопределённому поведению!
- Двумерный динамический массив (способ 1 — массив указателей int**): сначала выделяется массив из R указателей на строки: int** mat = new int*[rows];. Затем в цикле для каждой строки выделяется массив из C элементов: mat[i] = new int[cols];. Освобождение памяти выполняется СТРОГО В ОБРАТНОМ ПОРЯДКЕ: сначала в цикле delete[] mat[i]; для каждой строки, и только потом delete[] mat;.
- Двумерный динамический массив (способ 2 — плоский одномерный блок): выделяется один непрерывный буфер int* flat = new int[rows * cols];, а обращение к элементу (i, j) выполняется по формуле индексации: flat[i * cols + j]. Этот способ быстрее из-за кэш-локальности процессора и требует всего одного вызова delete[] flat;.

Пример кода C++: Создание, заполнение и корректное освобождение динамической 2D-матрицы (int**)

Полный цикл жизни динамического двумерного массива: двухэтапное выделение памяти, обращение через *(*(mat + i) + j) и двухэтапное удаление.

```
#include <iostream>

int main() {
    int rows = 3, cols = 4;

    // 1. Выделение памяти под динамическую матрицу (массив указателей н
    int** matrix = new int*[rows];
    for (int i = 0; i < rows; ++i) {
        matrix[i] = new int[cols];
```

```
    }

    // 2. Заполнение и вывод (демонстрация эквивалентности matrix[i][j]
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            *(*(matrix + i) + j) = (i + 1) * 10 + j; // то же, что matri
            std::cout << matrix[i][j] << " ";
        }
        std::cout << "
";
    }

    // 3. Освобождение памяти СТРОГО в обратном порядке!
    for (int i = 0; i < rows; ++i) {
        delete[] matrix[i]; // Сначала удаляем каждую строку
    }
    delete[] matrix;        // Затем удаляем массив указателей
    matrix = nullptr;
    return 0;
}
```

Результат выполнения (stdout):

```
10 11 12 13
20 21 22 23
30 31 32 33
```

Совет для экзамена: Самая частая ошибка на экзамене при удалении двумерного динамического массива — написать сначала delete[] matrix;, а потом пытаться в цикле делать delete[] matrix[i];! После удаления массива указателей обращаться к matrix[i] уже нельзя — это чтение освобождённой памяти.

24. Указатели-параметры. Массивы указателей.

Указатели в качестве параметров функций позволяют изменять внешние переменные, возвращать несколько результатов из функции и передавать крупные объекты без копирования. Массив указателей Тип* arr[N] хранит коллекцию адресов и позволяет эффективно сортировать данные без перемещения самих тяжёлых объектов.

Ключевые тезисы:

- Чтобы функция изменила значение обычной переменной, передают указатель T*; чтобы функция изменила сам внешний указатель (например, выделила под него память), передают указатель на указатель T** или ссылку на указатель T*&.
- Следует различать массив указателей int* arr[10] (массив из 10 указателей) и указатель на массив int (*ptr)[10] (один указатель на массив из 10 целых).
- Массивы указателей идеально подходят для хранения строк разной длины (рваных массивов — jagged arrays) и быстрой косвенной сортировки.

Указатели в качестве параметров функций

Передача параметров по указателю решает сразу три важные практические задачи:

- 1. Модификация аргумента и возврат нескольких значений (Out-параметры): так как оператор return возвращает только одно значение, через указатели в функцию передают адреса переменных, куда функция записывает дополнительные результаты (например, void minMax(const int a[], int n, int* outMin, int* outMax)).
- 2. Экономия памяти и времени при передаче больших структур/массивов: вместо копирования килобайтов данных в стек передаётся один 8-байтный указатель const BigStruct* p.
- 3. Изменение самого указателя внутри функции (T** или T*&): если функция должна выделить динамическую память и записать новый адрес во внешний

указатель вызывающего кода, недостаточно передать T* p (так как скопируется только адрес). Нужно передать адрес самого указателя T** pp или ссылку на указатель T*& rp.

Массивы указателей и их применение

Массив указателей объявляется как Тип* Имя[Размер]; (так как приоритет квадратных скобок [] выше, чем у звёздочки *, запись int* p[5]; читается как «массив p из 5 элементов, каждый из которых имеет тип int*»).

Основные сценарии использования массивов указателей:

- Хранение набора строк разной длины без перерасхода памяти: const char* months[12] = {"Январь", "Февраль", ...}; — в отличие от двумерной матрицы char m[12][20], каждая строка занимает ровно столько байт, сколько в ней символов.
- Ступенчатые (зубчатые / рваные) массивы (Jagged Arrays): двумерная структура, где каждая строка имеет свою собственную длину (например, треугольник Паскаля, где в i-й строке ровно i + 1 элементов).
- Индексная (косвенная) сортировка: если нужно отсортировать массив огромных объектов или длинных строк, вместо тяжёлого обмена самих объектов меняют местами лёгкие указатели на них в массиве указателей!

Пример кода C++: Указатели как выходные параметры функции и косвенная сортировка через массив указателей

Показывает возврат двух значений через указатели-параметры и сортировку массива указателей на строки без изменения исходного массива.

```
#include <iostream>
#include <cstring>

// Функция возвращает сразу два результата через указатели-параметры
void findMinAndMax(const int arr[], int n, int* outMin, int* outMax) {
    if (!outMin || !outMax || n <= 0) return;
    *outMin = arr[0];
    *outMax = arr[0];
    for (int i = 1; i < n; ++i) {
        if (arr[i] < *outMin) *outMin = arr[i];
        if (arr[i] > *outMax) *outMax = arr[i];
    }
}
```

```
int main() {
    int nums[] = {14, -3, 28, 7, 0};
    int mn = 0, mx = 0;
    findMinAndMax(nums, 5, &mn, &mx); // Передаём адреса переменных mn и
    std::cout << "Минимум = " << mn << ", Максимум = " << mx << "
";

    // Массив указателей на C-строки разной длины
    const char* langs[] = {"C++", "Ada", "Python", "Basic"};
    // Меняем местами только указатели (адреса), не копируя сами строки
    if (std::strcmp(langs[0], langs[1]) > 0) {
        const char* temp = langs[0];
        langs[0] = langs[1];
        langs[1] = temp;
    }
    std::cout << "После обмена первых двух указателей: "
              << langs[0] << ", " << langs[1] << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
Минимум = -3, Максимум = 28
После обмена первых двух указателей: Ada, C++
```

Совет для экзамена: Не путайте на экзамене две записи со скобками и без: int* a[10]; — это массив из 10 указателей на int (размер 10 * 8 = 80 байт), а int (*p)[10]; — это ОДИН указатель на массив из 10 элементов int (размер 8 байт)!

25. Передача одномерных массивов в качестве параметров. Передача двумерных массивов в качестве параметров.

Массивы в C++ никогда не передаются в функции по значению (не копируются целиком). Вместо этого в функцию передаётся указатель на начало массива, поэтому размер массива необходимо передавать отдельным параметром. При передаче статической 2D-матрицы количество столбцов обязано быть известно на этапе компиляции.

Ключевые тезисы:

- Три записи заголовка функции для одномерного массива абсолютно идентичны для компилятора: void f(int a[]), void f(int a[10]) и void f(int* a).
- Внутри функции sizeof(a) для параметра-массива вернёт размер указателя (8 байт), поэтому длину массива int n всегда передают вторым параметром!
- Для передачи статического двумерного массива в параметре ОБЯЗАТЕЛЬНО указывают число столбцов: void f(int mat[][COLS], int rows), чтобы компилятор мог вычислить шаг смещения строки.

Передача одномерных массивов в функцию

При вызове функции с одномерным массивом в качестве аргумента происходит автоматическое преобразование имени массива в указатель на его нулевой элемент (Array-to-pointer decay).

Поэтому для компилятора C++ следующие три прототипа функции являются ОДНОЙ И ТОЙ ЖЕ сигнатурой: void process(int* arr, int n);, void process(int arr[], int n); и void process(int arr[100], int n); (число 100 в квадратных скобках параметра просто игнорируется компилятором!).

Следствие 1: так как передаётся реальный адрес начала массива, любые изменения элементов arr[i] внутри функции изменяют исходный массив в вызывающем коде! Если функция не должна менять массив, параметр обязательно объявляют с квалификатором const: void print(const int arr[], int n).

Следствие 2: внутри функции нельзя узнать длину массива через sizeof(arr) / sizeof(arr[0]), поэтому количество элементов передают отдельным параметром int n (либо передают контейнер std::vector<int>& или std::array<int, N>&).

Способы передачи двумерных массивов в функцию

Способ передачи двумерного массива зависит от того, как именно эта матрица была создана в памяти (статически или динамически):

Тип матрицы в main()	**Как объявляется параметр функции**	**Почему это так работает**
Статическая матрица int m[3][4]	void f(const int m[][4], int rows) или void f(const int (*m)[4], int rows)	Матрица лежит единым блоком. Чтобы вычислить адрес m[i][j] = *(m + i*COLS + j), компилятор обязан знать ширину строки COLS (4) на этапе компиляции!
Динамический массив указателей int** dm	void f(int** dm, int rows, int cols)	Передаётся указатель на массив указателей на строки. Подходит для любых динамических размеров rows и cols.
Шаблон функции для любой статической матрицы	template <size_t R, size_t C> void f(const int (&m)[R][C])	Передача массива по ссылке (&m) предотвращает распад в указатель и автоматически выводит оба размера R и C.
Современный подход STL	void f(const std::vector<std::vector<int>>& m)	Безопасная передача по константной ссылке; размеры доступны через m.size() и m[0].size().

Пример кода C++: Передача одномерного массива и статической/ динамической 2D-матрицы в функции

Показывает передачу 1D-массива, статической матрицы с фиксированным числом столбцов и шаблонную передачу матрицы по ссылке.

```
#include <iostream>

// 1. Приём одномерного массива (const защищает оригинал от случайного и
int sum1D(const int arr[], int n) {
    int s = 0;
    for (int i = 0; i < n; ++i) s += arr[i];
    return s;
}

// 2. Приём статического двумерного массива: второе измерение (COLS = 3)
int sumStatic2D(const int mat[][3], int rows) {
    int total = 0;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < 3; ++j) {
            total += mat[i][j];
        }
    }
    return total;
}

// 3. Шаблонная передача 2D-массива по ссылке (принимает матрицы любых р
template <size_t R, size_t C>
void printMatrixDims(const int (&mat)[R][C]) {
    std::cout << "Размеры матрицы по ссылке: " << R << "x" << C
              << ", первый элемент = " << mat[0][0] << "
";
}

int main() {
    int vec[4] = {5, 10, 15, 20};
    int grid[2][3] = {{1, 2, 3}, {4, 5, 6}};

    std::cout << "Сумма 1D массива: " << sum1D(vec, 4) << "
";
    std::cout << "Сумма 2D матрицы: " << sumStatic2D(grid, 2) << "
";
    printMatrixDims(grid);
    return 0;
}
```

Результат выполнения (stdout):

```
Сумма 1D массива: 50
Сумма 2D матрицы: 21
```

```
Размеры матрицы по ссылке: 2x3, первый элемент = 1
```

Совет для экзамена: Запомните: статическую матрицу int grid[2][3] НЕЛЬЗЯ напрямую передать в функцию void f(int** m), потому что у статической матрицы в памяти нет массива указателей — там лежат сразу 6 чисел int подряд! Для статической матрицы нужен параметр int m[][3], а для динамического массива указателей — int** m.

26. Алгоритмы поиска и сортировки. Стандартные функции сортировки и поиска.

Базовые алгоритмы поиска включают линейный поиск O(n) и бинарный (двоичный) поиск O(log n) в отсортированном массиве. Среди сортировок различают квадратичные O(n^2) (пузырьком, выбором, вставками) и эффективные O(n log n) (быстрая сортировка Хоара, слиянием). В STL для них предоставлены std::sort, std::find и std::binary_search.

Ключевые тезисы:

- Линейный поиск работает на любом массиве за O(n); бинарный поиск требует ОТСОРТИРОВАННОГО массива и делит область поиска пополам на каждом шаге за O(log n).
- Квадратичные сортировки (Bubble, Selection, Insertion) имеют сложность O(n^2), а быстрая сортировка (QuickSort / std::sort) работает за O(n log n).
- В заголовке <algorithm> находятся стандартные алгоритмы std::sort, std::find, std::binary_search, std::lower_bound, а в <cstdlib> — C-функции qsort и bsearch.

Алгоритмы поиска: линейный и бинарный (двоичный)

1. Линейный (последовательный) поиск: поочерёдно проверяет элементы массива с 0-го до (n-1)-го. Не требует предварительной сортировки массива. Временная сложность в худшем и среднем случае — O(n).

2. Бинарный (двоичный / дихотомический) поиск: применяется ТОЛЬКО к упорядоченному (отсортированному) массиву. Поддерживаются границы поиска [left, right]. На каждом шаге вычисляется средний индекс mid = left + (right - left) / 2. Если arr[mid] == key, элемент найден; если arr[mid] < key, поиск продолжается в правой половине (left = mid + 1), иначе — в левой (right = mid - 1). Сложность — O(log2 n) (для 1 000 000 элементов нужно максимум 20 сравнений!).

Алгоритмы сортировки и стандартные функции в C++

Сортировка упорядочивает элементы по неубыванию или невозрастанию ключа:

- Стандартные функции <algorithm>: std::sort(v.begin(), v.end()) — быстрая сортировка; std::find(v.begin(), v.end(), val) — линейный поиск; std::binary_search(v.begin(), v.end(), val) — возвращает true/false за O(log n); std::lower_bound(v.begin(), v.end(), val) — возвращает итератор на первый элемент >= val за O(log n).

Алгоритм сортировки	**Идея работы**	**Сложность (Средняя / Худшая)**	**Устойчивость (Stability)**
Пузырьком (Bubble Sort)	Многократный проход по массиву с обменом соседних элементов, стоящих в неправильном порядке.	O(n^2) / O(n^2) (лучшая O(n) с флагом обмена)	Устойчивая (не меняет порядок равных)
Выбором (Selection Sort)	На i-м шаге ищется минимальный элемент в неотсортированной части [i..n-1] и меняется местами с a[i].	O(n^2) / O(n^2) (минимум обменов — не более n)	Неустойчивая
Вставками (Insertion Sort)	Очередной элемент вставляется на своё место среди уже отсортированных элементов слева.	O(n^2) / O(n^2) (очень быстрая на почти отсортированных данных)	Устойчивая
Быстрая сортировка (QuickSort) и std::sort	Выбирается опорный элемент (pivot), массив делится на меньшие и большие pivot, обе части сортируются рекурсивно (в STL — IntroSort).	O(n log n) / В std::sort гарантировано O(n log n)	std::sort неустойчивая (для устойчивой есть std::stable_sort)

Пример кода C++: Реализация бинарного поиска O(log n) и использование стандартных std::sort и std::lower_bound

Сортировка вектора стандартной функцией std::sort и поиск индекса элемента с помощью классического алгоритма бинарного поиска.

```
#include <iostream>
#include <vector>
#include <algorithm>

// Классическая реализация бинарного поиска (возвращает индекс или -1)
int binarySearchIndex(const std::vector<int>& arr, int target) {
    int left = 0;
    int right = static_cast<int>(arr.size()) - 1;

    while (left <= right) {
        // Защита от переполнения int вместо (left + right) / 2
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) {
            return mid; // Элемент найден!
        } else if (arr[mid] < target) {
            left = mid + 1;  // Ищем в правой половине
        } else {
            right = mid - 1; // Ищем в левой половине
        }
    }
    return -1; // Элемент отсутствует
}

int main() {
    std::vector<int> data = {45, 12, 89, 3, 27, 64, 19};

    // 1. Сортируем за O(N log N) стандартной функцией STL
    std::sort(data.begin(), data.end());

    std::cout << "Отсортированный массив: ";
    for (int x : data) std::cout << x << " ";
    std::cout << "
";

    // 2. Ищем элемент 27 с помощью нашего бинарного поиска и std::binar
    int idx = binarySearchIndex(data, 27);
    bool exists = std::binary_search(data.begin(), data.end(), 27);
    std::cout << "Ключ 27 найден на индексе: " << idx
              << " (std::binary_search: " << std::boolalpha << exists <<
";
```

```
    return 0;
}
```

Результат выполнения (stdout):

```
Отсортированный массив: 3 12 19 27 45 64 89
Ключ 27 найден на индексе: 3 (std::binary_search: true)
```

Совет для экзамена: В бинарном поиске всегда пишите вычисление середины как mid = left + (right - left) / 2 вместо (left + right) / 2. На больших массивах сумма left + right может превысить INT_MAX (~2.14 млрд) и вызвать переполнение знакового целого!

27. Строки. Операции над строками. Понятие нуль-терминированных строк.

В C++ существуют два представления строк: классические нуль-терминированные строки в стиле C (массивы символов char[], оканчивающиеся специальным нулевым байтом '\0' с кодом 0) и объектный класс std::string.

Ключевые тезисы:

- Нуль-терминированная C-строка длины L требует массив минимум из L + 1 байт (последний байт обязан хранить символ конца строки '\0').
- Строковый литерал "Hi" в двойных кавычках имеет тип const char[3] ('H', 'i', '\0'), тогда как 'A' в одинарных кавычках — это один символ типа char.
- Функции для работы с C-строками из заголовка <cstring>: strlen (длина без '\0'), strcpy/strncpy (копирование), strcat (конкатенация), strcmp (сравнение).

Понятие нуль-терминированной строки (C-style string)

Нуль-терминированная строка — это массив элементов типа char, в котором последовательность значимых символов завершается нулевым терминатором '\0' (символ с ASCII-кодом 0, не путать с цифрой '0', чей ASCII-код равен 48!).

Способы объявления и инициализации C-строки: 1) char str[] = "Hello"; — компилятор сам выделяет массив из 6 байт и дописывает '\0' в конец; 2) char str[10] = {'C', '+', '+', '\0'}; — при посимвольной инициализации программист обязан не забыть нулевой символ; 3) const char* ptr = "Literal"; — указатель на неизменяемый строковый литерал в памяти только для чтения (.rodata).

Стандартные операции над C-строками (библиотека <cstring>)

Так как C-строка технически является обычным массивом char[], к ней НЕЛЬЗЯ напрямую применять операторы присваивания s1 = s2, сложения s1 + s2 или сравнения по значению s1 == s2 (оператор == сравнит лишь адреса указателей!). Вместо этого используются функции из <cstring>:

Функция <cstring>	Назначение	Важные особенности
size_t strlen(const char* s)	Возвращает количество символов ДО первого '\0'.	Сам символ '\0' в длину strlen не входит (в отличие от sizeof). Сложность O(n).
char* strcpy(char* dest, const char* src)	Копирует строку src (включая '\0') в буфер dest.	Буфер dest обязан иметь достаточный размер, иначе произойдёт переполнение буфера (безопаснее strncpy).
char* strcat(char* dest, const char* src)	Присоединяет (конкатенирует) строку src в конец строки dest.	Находит '\0' в dest и перезаписывает его символами из src, ставя новый '\0' в конце.
int strcmp(const char* s1, const char* s2)	Лексикографическое (посимвольное) сравнение двух строк.	Возвращает 0, если строки РАВНЫ; < 0, если s1 меньше s2; > 0, если s1 больше s2.
const char* strchr(s, ch) / strstr(s, sub)	Поиск первого вхождения символа ch или подстроки sub.	Возвращает указатель на найденную позицию внутри строки s или nullptr, если не найдено.

Пример кода C++: Устройство нуль-терминированной строки и функции <cstring>

Сравнение strlen и sizeof, конкатенация через strcat и лексикографическое сравнение через strcmp.

```
#include <iostream>
#include <cstring>

int main() {
    // Строка из 5 видимых букв занимает 6 байт в памяти ('H','e','l','l
    char word[] = "Hello";
    std::cout << "strlen(word) = " << std::strlen(word)
              << ", sizeof(word) = " << sizeof(word) << " байт
```

```
";

    // Буфер с запасом места для конкатенации
    char buffer[32] = "C++ ";
    std::strcat(buffer, "Programming"); // Присоединяем вторую строку
    std::cout << "После strcat: " << buffer << "
";

    // Правильное сравнение содержимого C-строк через strcmp (0 означает
    const char* a = "Apple";
    const char* b = "Apricot";
    if (std::strcmp(a, b) < 0) {
        std::cout << """ << a << "" идет по алфавиту раньше, чем "" << b
";
    }
    return 0;
}
```

Результат выполнения (stdout):

```
strlen(word) = 5, sizeof(word) = 6 байт
После strcat: C++ Programming
"Apple" идет по алфавиту раньше, чем "Apricot"
```

Совет для экзамена: Обязательно помните: условие if (strcmp(s1, s2)) проверяет, что строки РАЗНЫЕ (так как ненулевое число в bool — это true)! Чтобы проверить равенство двух C-строк, всегда пишите: if (std::strcmp(s1, s2) == 0).

28. Функция main. Передача параметров в функцию main.

Функция main() — обязательная точка входа в любую стандартную программу на C++. Для приёма аргументов командной строки от операционной системы используется сигнатура int main(int argc, char* argv[]), где argc — счётчик аргументов, а argv — массив строк-аргументов.

Ключевые тезисы:

- По стандарту C++ функция main() ОБЯЗАНА иметь тип возвращаемого значения int (0 или EXIT_SUCCESS означает успешное завершение).
- Параметр argc (argument count) всегда >= 1, так как нулевой аргумент argv[0] содержит имя (или путь) самого запущенного исполняемого файла.
- Все аргументы командной строки передаются в argv[] как текстовые C-строки (char*); для перевода в числа используют std::stoi, std::stod или std::atoi.

Роль функции main() и коды возврата операционной системе

Каждая программа на C++ (в размещённой среде с ОС) должна содержать ровно одну глобальную функцию с именем main. Именно с неё начинается выполнение пользовательского кода после инициализации глобальных переменных.

Стандарт C++ регламентирует две канонические сигнатуры функции main: 1) int main() — без параметров; 2) int main(int argc, char* argv[]) (или эквивалентно int main(int argc, char** argv)) — с параметрами командной строки.

Особенность main(): это единственная не-void функция в C++, в самом конце которой разрешено не писать инструкцию return 0; — компилятор подставит возврат нуля автоматически. Возврат 0 (или макроса EXIT_SUCCESS из <cstdlib>) сообщает операционной системе или скрипту запуска, что программа отработала без ошибок; ненулевой код (например, 1 или EXIT_FAILURE) сигнализирует об ошибке.

Разбор параметров командной строки: argc и argv

Когда пользователь запускает программу из терминала командой ./app.exe input.txt 42, операционная система разбивает строку по пробелам и передаёт аргументы в main:

- int argc (Argument Count): общее количество переданных токенов, включая само имя программы. Если программа запущена вообще без дополнительных параметров, argc равен 1.
- char* argv[] (Argument Vector): массив указателей на нуль-терминированные строки. Элемент argv[0] — путь/имя самой программы ("./app.exe"); элементы с argv[1] по argv[argc - 1] — переданные пользователем параметры; а элемент argv[argc] по стандарту гарантированно равен nullptr.
- Если аргумент содержит пробелы, при запуске в консоли его заключают в двойные кавычки: ./app "Мой файл.txt" — тогда он попадёт в один элемент argv[1].

Пример кода C++: Обработка аргументов командной строки argc и argv с конвертацией в числа

Показывает, как программа разбирает переданные ей параметры командной строки и складывает два числа.

```
#include <iostream>
#include <string>

// Точка входа с параметрами командной строки
int main(int argc, char* argv[]) {
    // Симуляция аргументов для демонстрации, если запущено без параметр
    const char* demoArgs[] = {"./calc_app", "125", "75", nullptr};
    int count = (argc > 1) ? argc : 3;
    const char** args = (argc > 1) ? const_cast<const char**>(argv) : de

    std::cout << "Всего аргументов (argc): " << count << "
";
    for (int i = 0; i < count; ++i) {
        std::cout << "  argv[" << i << "] = "" << args[i] << ""
";
    }

    if (count >= 3) {
        // Аргументы приходят как текст (char*), переводим их в int чере
        int a = std::stoi(args[1]);
        int b = std::stoi(args[2]);
```

```
        std::cout << "Сумма аргументов: " << a << " + " << b << " = " <<
";
    }
    return 0; // Код успешного завершения для ОС
}
```

Результат выполнения (stdout):

```
Всего аргументов (argc): 3
  argv[0] = "./calc_app"
  argv[1] = "125"
  argv[2] = "75"
Сумма аргументов: 125 + 75 = 200
```

Совет для экзамена: В C++ (в отличие от старых диалектов C) категорически запрещено: 1) объявлять void main(), 2) вызывать функцию main() рекурсивно из самой программы или брать её адрес &main.

БИЛЕТ №29 · СТРОКИ И СТРУКТУРЫ

29. Класс string. Функции стандартной библиотеки для обработки строк.

Класс std::string (заголовок <string>) — безопасная динамическая строка стандартной библиотеки C++, которая автоматически управляет памятью, хранит свою длину за O(1) и поддерживает интуитивные операторы +, +=, =, ==, <.

Ключевые тезисы:

- В отличие от char[], std::string автоматически расширяет буфер при конкатенации (+, +=) и защищает от переполнения буфера.
- Для чтения всей строки вместе с пробелами из потока используют std::getline(std::cin, str), так как оператор std::cin >> str читает только до первого пробела!
- Ключевые методы: size()/length(), substr(pos, len), find(sub), replace(pos, len, newStr), erase(pos, len), insert(pos, str), c_str() и конвертации std::to_string / std::stoi.

Преимущества класса std::string и базовые операции

Класс std::string (специализация шаблона std::basic_string<char> из заголовка <string>) инкапсулирует динамический массив символов. Он избавляет программиста от ручного выделения памяти и ошибок с забытым нулевым терминатором '\0'.

Перегруженные операторы делают работу со строками естественной: присваивание (s1 = s2), склеивание/конкатенация (s3 = s1 + " " + s2; s1 += "!";), прямое лексикографическое сравнение содержимого (s1 == s2, s1 < s2) и доступ к символам по индексу s[i] или с проверкой границ s.at(i).

Основные методы std::string и функции конвертации

Стандартная библиотека предоставляет богатый набор методов для поиска, извлечения подстрок и модификации текста:

Метод / Функция	Описание и результат	Пример вызова
s.size() / s.length()	Возвращает текущее число символов за время O(1) (методы-синонимы).	size_t n = s.length();
s.empty()	Возвращает true, если строка пустая (длина равна 0).	if (!s.empty()) { ... }
s.substr(pos, count)	Возвращает новую подстроку, начиная с индекса pos длины count.	string sub = s.substr(0, 4);
s.find(pattern, pos)	Ищет первое вхождение подстроки. Если не найдено, возвращает константу std::string::npos!	if (s.find("key") != string::npos)
s.replace(pos, len, str)	Заменяет участок строки [pos, pos+len) на новую строку str.	s.replace(0, 3, "New");
s.erase(pos, len) / s.insert(pos, str)	Удаляет len символов с позиции pos / вставляет строку str в позицию pos.	s.erase(2, 4);
s.c_str()	Возвращает константный указатель const char* на нуль-терминированный буфер для совместимости с C-функциями.	printf("%s", s.c_str());
std::to_string(val) / std::stoi(s), std::stod(s)	Преобразует число в std::string / парсит строку в int или double.	int x = std::stoi("42");

Пример кода C++: Поиск, замена, выделение подстроки и конвертация чисел с std::string

Комплексный пример обработки текста с проверкой результата поиска через std::string::npos.

```
#include <iostream>
#include <string>

int main() {
    std::string text = "Стандарт языка C++98 устарел";

    // 1. Поиск подстроки и проверка через std::string::npos
    size_t pos = text.find("C++98");
```

```
    if (pos != std::string::npos) {
        // Заменяем 5 символов ("C++98") на "C++20"
        text.replace(pos, 5, "C++20");
    }

    // 2. Извлечение подстроки и конкатенация с числом через std::to_str
    std::string lang = text.substr(pos, 5); // "C++20"
    int year = std::stoi(lang.substr(3, 2)) + 2000; // "20" -> 2020

    std::cout << "Обновлённый текст: " << text << "
";
    std::cout << "Извлечённый стандарт: " << lang << " (год: " << year <
";
    std::cout << "Длина строки в байтах: " << text.length() << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
Обновлённый текст: Стандарт языка C++20 устарел
Извлечённый стандарт: C++20 (год: 2020)
Длина строки в байтах: 50
```

Совет для экзамена: Ловушка при смешивании std::cin >> n; и std::getline(std::cin, str);: оператор >> оставляет в буфере ввода символ перевода строки '\n', и следующий за ним getline мгновенно считывает пустую строку! Чтобы пропустить пробельные символы перед getline, используйте std::getline(std::cin >> std::ws, str); или std::cin.ignore().

БИЛЕТ №30 · СТРОКИ И СТРУКТУРЫ

30. Пользовательские типы данных. Перечислимый тип enum. Тип struct. Массивы структур.

Пользовательские типы данных позволяют моделировать сущности предметной области. Перечисление (enum и строго типизированное enum class) задаёт набор именованных целочисленных констант, а структура (struct) объединяет разнородные поля под одним именем.

Ключевые тезисы:

- В классическом enum константы попадают в окружающую область видимости и неявно приводятся к int; в современном enum class (C++11) имена изолированы (Color::Red) и требуют явного static_cast.
- Структура (struct) объединяет переменные разных типов (поля/члены структуры) в единый составной тип.
- Доступ к полям объекта структуры или элемента массива структур выполняется через точку (student.gpa, group[i].name), а через указатель на структуру — с помощью оператора стрелки (ptr->name).

Перечислимый тип: классический enum и строгий enum class

Перечисление (Enumeration) — пользовательский целочисленный тип, значения которого ограничены списком именованных констант (энумераторов). По умолчанию первая константа получает значение 0, а каждая следующая увеличивается на 1 (можно задать стартовые значения явно, например Red = 1, Green = 5).

Характеристика	Классический enum (C-style)	Строгое перечисление enum class (C++11)
Синтаксис объявления	enum Status { Ok, Error };	enum class Status { Ok, Error };

Характеристика	Классический enum (C-style)	Строгое перечисление enum class (C++11)
Область видимости имён	Загрязняет внешнюю область (нельзя иметь два enum с константой Ok).	Имена изолированы внутри перечисления: Status::Ok.
Неявное приведение к int	Да (можно случайно сравнить Цвет со ДнёмНедели).	Запрещено! Для получения числа нужен static_cast<int>(Status::Ok).

Структуры (struct) и массивы структур

В отличие от массива, где все элементы обязаны быть одного типа, структура (struct) позволяет объединить в один объект логически связанные данные РАЗНЫХ типов (например, карточка студента: фамилия string, курс int, средний балл double). В C++ ключевое слово struct создаёт полноценный тип данных (все поля по умолчанию имеют спецификатор доступа public).

- Обращение к полям структуры: 1) через оператор «точка» (.) для обычной переменной или ссылки: s.gpa = 4.8;; 2) через оператор «стрелка» (->) для указателя на структуру: ptr->gpa = 4.8; (что полностью эквивалентно записи (*ptr).gpa = 4.8;).
- Операции со структурами: в отличие от массивов, структуры одного типа МОЖНО присваивать друг другу целиком (s2 = s1; — выполняет почленное копирование всех полей), передавать в функции (обычно по константной ссылке const Student& s) и возвращать из функций по значению.
- Массивы структур: объявление Student group[30]; или std::vector<Student> group; создаёт таблицу записей, где обращение group[i].name выбирает сначала i-ю структуру в массиве, а затем её поле name.

Пример кода C++: Использование enum class, struct, указателя (->) и сортировки массива структур

Моделирование ведомости студентов с перечислимым типом формы обучения и фильтрацией массива структур.

```
#include <iostream>
#include <string>
#include <vector>
```

```
// Строго типизированное перечисление
enum class StudyForm { Budget, Contract };

// Определение пользовательской структуры
struct Student {
    std::string name;
    int course;
    double gpa;
    StudyForm form;
};

// Передача структуры по константной ссылке (без лишнего копирования)
void printStudent(const Student* sPtr) {
    // Доступ к полям через указатель с помощью оператора стрелки ->
    std::cout << sPtr->name << " (курс " << sPtr->course
              << ", балл: " << sPtr->gpa << ", "
              << (sPtr->form == StudyForm::Budget ? "Бюджет" : "Контракт
";
}

int main() {
    // Массив (вектор) структур с агрегатной инициализацией
    std::vector<Student> group = {
        {"Иванов А.С.", 1, 4.85, StudyForm::Budget},
        {"Петрова М.В.", 1, 3.90, StudyForm::Contract},
        {"Смирнов Д.К.", 2, 4.95, StudyForm::Budget}
    };

    std::cout << "Отличники (GPA >= 4.5):
";
    for (size_t i = 0; i < group.size(); ++i) {
        if (group[i].gpa >= 4.5) { // Обращение к элементу массива струк
            printStudent(&group[i]);
        }
    }
    return 0;
}
```

Результат выполнения (stdout):

```
Отличники (GPA >= 4.5):
Иванов А.С. (курс 1, балл: 4.85, Бюджет)
Смирнов Д.К. (курс 2, балл: 4.95, Бюджет)
```

Совет для экзамена: На экзамене могут спросить, почему sizeof(Struct) иногда оказывается больше суммы размеров всех её полей. Ответ: из-за выравнивания данных в памяти (Data

Alignment / Padding) — компилятор вставляет пустые байты-выравниватели, чтобы 4- и 8-байтные поля начинались по адресам, кратным 4 или 8.

БИЛЕТ №31 · СТРОКИ И СТРУКТУРЫ

31. Типы ошибок и их обработка на этапах проектирования, трансляции, выполнения. Отладка программ в среде разработки. Степпинг, точки останова. Стек вызовов и отслеживание переменных.

Ошибки в программировании делятся на ошибки проектирования/логические (алгоритмические), ошибки трансляции (компиляции и компоновки) и ошибки времени выполнения (Runtime errors). Для их поиска и устранения применяется интерактивный отладчик (Debugger) с точками останова, пошаговым выполнением (Stepping) и анализом стека вызовов.

Ключевые тезисы:

- Ошибки компиляции (синтаксические и семантические) и ошибки компоновки (Linker errors) не дают собрать исполняемый файл.
- Ошибки времени выполнения (деление на 0, выход за границы массива) и логические ошибки проявляются уже в работающей программе.
- Три режима пошаговой отладки (Stepping): Step Into (F11 — зайти внутрь функции), Step Over (F10 — выполнить функцию как один шаг), Step Out (Shift+F11 — довыполнить текущую функцию до возврата).

Классификация ошибок по этапам жизненного цикла программы

В зависимости от момента обнаружения и природы дефекта выделяют четыре основные категории ошибок:

Категория ошибок	На каком этапе возникает	Примеры и симптомы
1. Ошибки проектирования и	Этап проектирования и	Программа успешно компилируется и не падает, но

Категория ошибок	На каком этапе возникает	Примеры и симптомы
логические (алгоритмические)	кодирования (выявляются при тестировании)	выдаёт НЕВЕРНЫЙ ответ (например, перепутан знак < вместо >, неверная формула дискриминанта, зацикливание). Самые трудные для поиска!
2. Ошибки компиляции (Syntax & Semantic Errors)	Этап трансляции (компилятор)	Нарушение грамматики или системы типов: пропущена ;, необъявленная переменная, попытка присвоить строку в int.
3. Ошибки компоновки (Linker Errors)	Этап сборки объектных файлов в .exe (линковщик)	Unresolved external symbol (есть прототип функции, но нет её реализации), Multiple definition (две функции с одним именем), отсутствие main().
4. Ошибки времени выполнения (Runtime Errors)	Во время работы запущенной программы	Деление целого числа на 0, разыменование nullptr (Segmentation fault / Access violation), нехватка памяти в куче, отсутствие открываемого файла на диске.

Инструменты отладки в IDE: точки останова, степпинг, стек вызовов и Watch

Отладка (Debugging) — процесс локализации и устранения причины уже обнаруженной ошибки. Современные среды разработки (Visual Studio, CLion, VS Code + GDB/LLDB) предоставляют мощный интерактивный отладчик:

- Точка останова (Breakpoint, клавиша F9): специальная метка на строке исходного кода, при достижении которой выполнение программы ставится на паузу, позволяя разработчику изучить текущее состояние памяти. Бывают условные точки останова (Conditional Breakpoint — срабатывают только когда, например, i == 99).
- Степпинг (Stepping — пошаговое выполнение): управление программой в состоянии паузы по одной строчке: 1) Шаг с заходом (Step Into / F11) — если на текущей строке есть вызов функции, отладчик входит внутрь её тела; 2) Шаг с обходом (Step Over / F10) — выполняет вызываемую функцию целиком

как одну команду и останавливается на следующей строке текущей функции; 3) Шаг с выходом (Step Out / Shift+F11) — выполняет оставшуюся часть текущей функции и останавливается сразу после возврата из неё.

- Стек вызовов (Call Stack): окно отладчика, показывающее цепочку вложенных вызовов функций, которая привела программу в текущую точку (например: main() -> processArray() -> validateItem()), с возможностью переключиться на любой кадр стека.
- Отслеживание переменных (Locals / Autos / Watch): окна для просмотра и изменения в реальном времени значений локальных переменных и произвольных выражений (например, arr[i] * 2) на каждом шаге.

Пример кода C++: Отладочные утверждения assert() и диагностика логических ошибок

Использование макроса assert из <cassert> для автоматической проверки инвариантов в режиме отладки (Debug) и предотвращения ошибок времени выполнения.

```
#include <iostream>
#include <cassert>

double calculateAverage(const int arr[], int size) {
    // Отладочная проверка предусловия: массив должен существовать, а ра
    assert(arr != nullptr && "Указатель на массив не должен быть nullptr
    assert(size > 0 && "Размер массива должен быть строго положительным!

    long long sum = 0;
    // Типичная логическая ошибка новичка: i <= size (выход за границу н
    // Исправленное условие: i < size
    for (int i = 0; i < size; ++i) {
        sum += arr[i];
    }
    // Явное приведение к double предотвращает логическую ошибку целочис
    return static_cast<double>(sum) / size;
}

int main() {
    int grades[] = {4, 5, 5, 3, 5};
    double avg = calculateAverage(grades, 5);
    std::cout << "Корректный средний балл: " << avg << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
Корректный средний балл: 4.4
```

Совет для экзамена: На экзамене чётко разделяйте Step Over (F10) и Step Into (F11): если курсор стоит на строке y = calc(x);, то F11 нырнёт на первую строчку внутрь функции calc, а F10 мгновенно вычислит calc(x) и перешагнёт на следующую строчку после вызова.

32. Файлы. Основные принципы работы с файлами. Механизм чтения данных из файла. Определение конца файла. Открытие и закрытие файлов. Последовательный и прямой метод доступа.

Файл — именованная область внешней энергонезависимой памяти. В C++ работа с файлами строится через файловые потоки библиотеки <fstream> (ifstream для чтения, ofstream для записи, fstream для чтения и записи) по схеме: открытие → проверка успешности → чтение/запись → закрытие.

Ключевые тезисы:

- Стандартный жизненный цикл работы с файлом: 1) привязка потока и открытие файла, 2) проверка is_open(), 3) обмен данными, 4) закрытие close() (вызывается также автоматически в деструкторе).
- Самый надёжный способ чтения до конца файла в C++ — проверка самого выражения чтения в условии цикла: while (in >> value) или while (std::getline(in, line)).
- Последовательный доступ читает данные строго от начала к концу байт за байтом, а прямой (произвольный) доступ позволяет мгновенно перемещать файловый указатель в любую позицию через seekg/seekp.

Основные принципы открытия, проверки и закрытия файлов в C++

Оперативная память (ОЗУ) очищается при завершении программы. Для долговременного хранения данных используются файлы на внешних носителях. В C++ за работу с файлами отвечает заголовочный файл <fstream>, предоставляющий три класса: std::ifstream (input file stream — чтение), std::ofstream (output file stream — запись) и std::fstream (двунаправленный поток).

Открыть файл можно сразу в конструкторе std::ifstream in("data.txt"); либо вызовом метода in.open("data.txt", mode);. Обязательным правилом хорошего тона является проверка успешности открытия через if (!in.is_open()).

Режимы открытия файла (битовые флаги std::ios, комбинируются через побитовое ИЛИ |): ios::in (чтение), ios::out (запись с перезаписью файла с нуля), ios::app (append — дозапись строго в конец файла), ios::ate (переход в конец при открытии), ios::trunc (очистка существующего содержимого), ios::binary (двоичный режим без преобразования спецсимволов).

Механизм чтения данных, определение конца файла (EOF) и методы доступа

При чтении из файла операционная система и поток поддерживают внутренний указатель текущей позиции чтения (get pointer). После каждой операции чтения он автоматически сдвигается вперёд.

- Почему цикл while (!in.eof()) { in >> x; process(x); } — это классическая ошибка? Потому что флаг конца файла eof() устанавливается ТОЛЬКО ПОСЛЕ неудачной попытки чтения за концом файла! В результате последняя прочитанная переменная обработается в цикле ДВАЖДЫ. Правильный идиоматический паттерн C++: помещать само чтение в условие цикла: while (in >> x) { process(x); }.
- Последовательный метод доступа: данные считываются или записываются строго по порядку от первого байта к последнему (чтобы прочитать 100-ю запись, нужно сначала пройти первые 99). Типичен для текстовых файлов с строками разной длины.
- Прямой (произвольный / Random Access) метод доступа: позволяет мгновенно за O(1) переставить указатель файла на любой байт с помощью методов seekg / seekp и сразу прочитать нужную запись (особенно эффективен в бинарных файлах со структурами фиксированного размера).

Пример кода C++: Полный цикл работы с файловым потоком: запись, проверка is_open и чтение до EOF

Демонстрирует корректный шаблон записи данных в поток и безопасного чтения до конца файла без дублирования последней строки.

```
#include <iostream>
#include <fstream>
```

```
#include <sstream>
#include <string>

int main() {
    // В примере используем строковый поток (интерфейс полностью идентич
    std::stringstream fileStream;

    // 1. Запись данных в поток
    fileStream << "100 250 400
";
    fileStream << "550 700
";

    // 2. Правильный механизм чтения всех чисел до конца файла (EOF)
    int number = 0;
    int sum = 0;
    int count = 0;

    // Выражение (fileStream >> number) возвращает сам поток, который пр
    // пока чтение успешно, и автоматически становится false при достиже
    while (fileStream >> number) {
        sum += number;
        ++count;
    }

    std::cout << "Прочитано чисел: " << count << ", их сумма: " << sum <
";
    std::cout << "Достигнут ли конец потока (eof): " << std::boolalpha <
";
    return 0;
}
```

Результат выполнения (stdout):

```
Прочитано чисел: 5, их сумма: 2000
Достигнут ли конец потока (eof): true
```

Совет для экзамена: Запомните на экзамене: при открытии файла через std::ofstream out("log.txt"); старое содержимое файла по умолчанию СТИРАЕТСЯ (подразумевается флаг ios::out | ios::trunc). Чтобы дописывать данные в конец существующего файла, используйте флаг std::ios::app: std::ofstream out("log.txt", std::ios::app);.

33. Понятие потока ввода-вывода. Операции над потоками. Классификация потоков ввода-вывода. Иерархия потоковых классов.

Поток ввода-вывода (Stream) в C++ — это универсальная объектно-ориентированная абстракция последовательности байтов, передаваемых между программой и внешним источником/приёмником (консолью, файлом, строкой в памяти).

Ключевые тезисы:

- Стандартные глобальные объекты потоков в <iostream>: std::cin (ввод), std::cout (буферизованный вывод), std::cerr (небуферизованный вывод ошибок) и std::clog (буферизованный лог).
- Потоки классифицируются по направлению (ввод, вывод, двунаправленные) и по привязанному устройству (консольные, файловые, строковые).
- Вершиной иерархии классов является ios_base → basic_ios, от которого наследуются istream и ostream, объединяющиеся в iostream.

Понятие потока, буферизация и классификация потоков

Поток (Stream) позволяет программисту работать с совершенно разными физическими устройствами (клавиатурой, экраном, жёстким диском, строкой в ОЗУ) с помощью одного и того же набора операторов (<< и >>) и методов. Потоки используют буферизацию (промежуточный буфер streambuf в ОЗУ), чтобы минимизировать количество медленных системных обращений к ОС.

- 1. Стандартные (консольные) потоки (<iostream>): std::cin (связан со стандартным вводом stdin), std::cout (стандартный вывод stdout), std::cerr (вывод ошибок stderr без буферизации — текст появляется мгновенно даже при аварийном сбое), std::clog (буферизованный поток журналирования).
- 2. Файловые потоки (<fstream>): std::ifstream, std::ofstream, std::fstream — работают с файлами на диске.

- 3. Строковые потоки в памяти (<sstream>): std::istringstream, std::ostringstream, std::stringstream — позволяют читать и форматировать данные прямо внутри объекта std::string в оперативной памяти.

Иерархия потоковых классов и флаги состояния потока

Все потоковые классы в C++ построены в единую иерархию наследования (шаблоны с приставкой basic_ для типа символа char или wchar_t):

- Методы проверки состояния потока: s.good() — всё отлично (нет ошибок); s.eof() — достигнут конец данных; s.fail() — нефатальная ошибка формата (например, ждали int, а ввели буквы "abc"); s.bad() — критическая аппаратная ошибка потери буфера. Сбросить ошибку и вернуть поток в рабочее состояние позволяет метод s.clear().

Класс в иерархии	Заголовок	Роль в архитектуре ввода-вывода
std::ios_base	<ios>	Базовый нешаблонный класс: хранит флаги форматирования (dec, hex, fixed), точность (precision) и ширину поля (width).
std::ios (basic_ios<char>)	<ios>	Наследник ios_base: хранит указатель на буфер streambuf и 4 флага состояния потока (goodbit, eofbit, failbit, badbit).
std::istream / std::ostream	<istream> / <ostream>	Классы входных (оператор >>, get, getline, read) и выходных (оператор <<, put, write, flush) потоков.
std::iostream	<iostream>	Множественное (ромбовидное виртуальное) наследование от istream и ostream: поддерживает одновременно и ввод, и вывод.
ifstream / ofstream / fstream	<fstream>	Наследники istream, ostream и iostream для работы с файлами.
istringstream / ostringstream / stringstream	<sstream>	Наследники istream, ostream и iostream для работы со строками в памяти.

Пример кода C++: Единообразие работы с потоками (std::ostream&) и восстановление потока после ошибки формата

Показывает полиморфную функцию, способную писать в любой выходной поток (cout, файл, stringstream), и очистку флага ошибки через clear().

```
#include <iostream>
#include <sstream>

// Функция принимает базовый класс std::ostream& — работает и с cout, и
void writeReport(std::ostream& out, const char* title, int value) {
    out << "[Отчёт] " << title << " = " << value << "
";
}

int main() {
    // 1. Вывод в консоль и в строковый поток одной и той же функцией
    std::ostringstream memStream;
    writeReport(std::cout, "Температура", 23);
    writeReport(memStream, "Давление", 760);
    std::cout << "Содержимое строкового потока: " << memStream.str();

    // 2. Обработка ошибки формата ввода (failbit) и восстановление пото
    std::istringstream input("invalid_text 42");
    int num = 0;
    if (!(input >> num)) {
        std::cout << "Ошибка чтения числа (fail = " << input.fail() << "
";
        input.clear(); // Сбрасываем флаг ошибки failbit
        std::string skippedWord;
        input >> skippedWord; // Пропускаем некорректное слово
        input >> num;         // Теперь успешно считываем 42!
    }
    std::cout << "Успешно прочитано число после восстановления: " << num
";
    return 0;
}
```

Результат выполнения (stdout):

```
[Отчёт] Температура = 23
Содержимое строкового потока: [Отчёт] Давление = 760
Ошибка чтения числа (fail = 1). Очищаем поток...
Успешно прочитано число после восстановления: 42
```

Совет для экзамена: Почему при наследовании iostream от istream и ostream в стандартной библиотеке используется ВИРТУАЛЬНОЕ наследование (virtual public basic_ios)? Чтобы избежать «проблемы ромба» и не дублировать два экземпляра базового класса basic_ios в одном объекте iostream!

34. Реализация форматированного ввода-вывода для потоковых классов. Флаги форматирования. Форматирующие методы. Манипуляторы потоков ввода-вывода и их использование.

Форматированный ввод-вывод преобразует внутреннее двоичное представление чисел и типов в человекочитаемый текст и обратно. Управление форматом (системой счисления, выравниванием, точностью дробной части, шириной поля) осуществляется через флаги ios, методы потока или потоковые манипуляторы (<iomanip>).

Ключевые тезисы:

- Операторы << и >> выполняют форматированный ввод-вывод. Все настройки формата (кроме ширины поля width / setw) действуют постоянно, пока их не отменят!
- Ширина поля вывода (s.width(N) или манипулятор std::setw(N)) сбрасывается в 0 сразу после вывода ОДНОГО следующего значения.
- Для вывода вещественных чисел с ровно K знаками после точки используют комбинацию std::fixed << std::setprecision(K) из заголовка <iomanip>.

Флаги форматирования и форматирующие методы класса ios_base

Каждый поток хранит битовую маску флагов форматирования типа fmtflags. Управлять ими можно напрямую с помощью методов потока:

- Методы управления флагами: s.setf(flags) — устанавливает флаги (для взаимоисключающих групп, таких как система счисления basefield, используют перегрузку с двумя аргументами: cout.setf(ios::hex, ios::basefield)); s.unsetf(flags) — сбрасывает флаги.

- Основные флаги std::ios: boolalpha (вывод true/false вместо 1/0), showpos (печатать знак + перед положительными числами), uppercase (заглавные буквы в hex и экспоненте), dec / oct / hex (система счисления), showbase (префиксы 0x и 0), fixed (фиксированная точка), scientific (экспоненциальная форма 1.23e+04), left / right / internal (выравнивание в поле).
- Форматирующие методы: s.width(n) — минимальная ширина поля вывода (действует ТОЛЬКО на один ближайший вывод!); s.precision(n) — точность вещественных чисел; s.fill(ch) — символ-заполнитель пустых позиций (по умолчанию пробел).

Манипуляторы потоков ввода-вывода (<iostream> и <iomanip>)

Вызывать методы cout.setf(...) и cout.width(...) между выводами громоздко. Поэтому в C++ созданы манипуляторы — специальные функции и объекты, которые вставляются прямо в цепочку операторов << или >>:

Манипулятор	Заголовок	Действие в потоке
std::endl / std::flush	<iostream>	Перевод строки '\n' со сбросом буфера / принудительный сброс буфера.
std::dec, std::hex, std::oct	<iostream>	Переключение системы счисления для целых чисел (10, 16, 8).
std::boolalpha / std::noboolalpha	<iostream>	Текстовый вывод логических значений (true / false) вместо (1 / 0).
std::fixed / std::scientific	<iostream>	Фиксированный формат (3.14) / научный формат (3.14e+00) для double.
std::setw(int n)	<iomanip>	Задаёт ширину поля n ТОЛЬКО для следующего выводимого элемента!
std::setprecision(int n)	<iomanip>	Задаёт общее число значащих цифр (или число знаков после запятой при включённом std::fixed).
std::setfill(char c)	<iomanip>	Устанавливает символ заполнения пустого пространства в поле (например, '0' или '.').

Пример кода C++: Форматированный вывод таблицы с помощью манипуляторов <iomanip>

Вывод выровненной таблицы чисел в десятичной и шестнадцатеричной системах и цен с фиксированной точностью 2 знака после запятой.

```
#include <iostream>
#include <iomanip>

int main() {
    double price1 = 19.5, price2 = 1250.378;
    int code1 = 42, code2 = 255;

    // Настраиваем фиксированный формат с 2 знаками после точки для цен
    std::cout << std::fixed << std::setprecision(2);

    // Печатаем красивую выровненную таблицу (setw нужно указывать перед
    std::cout << std::left  << std::setw(12) << "Товар"
              << std::right << std::setw(8)  << "Код(HEX)"
              << std::setw(12) << "Цена" << "
";
    std::cout << std::setfill('-') << std::setw(32) << "" << std::setfil
";

    std::cout << std::left  << std::setw(12) << "Кабель"
              << std::right << std::setw(8)  << std::hex << std::upperca
              << std::dec   << std::setw(12) << price1 << "
";

    std::cout << std::left  << std::setw(12) << "Адаптер"
              << std::right << std::setw(8)  << std::hex << code2
              << std::dec   << std::setw(12) << price2 << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
Товар       Код(HEX)        Цена
--------------------------------
Кабель          0X2A       19.50
Адаптер         0XFF     1250.38
```

Совет для экзамена: Запомните ключевое отличие std::setprecision(3) без fixed и с fixed: без флага fixed манипулятор setprecision(3) задаёт 3 ЗНАЧАЩИЕ цифры всего числа (число 12.3456 выведется как 12.3), а в паре std::fixed << std::setprecision(3) он задаёт ровно 3 цифры ПОСЛЕ десятичной точки (12.346)!

35. Реализация неформатированного ввода-вывода для потоковых классов.

Неформатированный ввод-вывод передаёт посимвольные или побайтовые блоки данных напрямую без пропуска пробелов и без преобразования типов в текстовое представление. Основные методы: get(), put(), getline(), read(), write(), gcount(), peek() и ignore().

Ключевые тезисы:

- В отличие от оператора >>, который пропускает все ведущие пробелы, табуляции и переводы строк, неформатированные методы get() и read() считывают КАЖДЫЙ байт как есть.
- Парные методы блочного побайтового обмена: in.read(char* buf, streamsize n) и out.write(const char* buf, streamsize n).
- Метод in.gcount() возвращает точное количество байт, фактически прочитанных последней неформатированной операцией ввода.

Посимвольный и построчный неформатированный ввод-вывод

Когда требуется прочитать текст с сохранением всех пробелов, знаков табуляции и пустых строк или обработать поток побайтово, форматированный оператор >> не подходит. Вместо него используются методы неформатированного обмена:

- in.get(char& ch) (или int ch = in.get()): считывает из потока ровно один любой символ (включая пробел и '\n'). Версия без аргументов возвращает код символа типа int или константу EOF (-1) при конце файла.
- out.put(char ch): записывает в выходной поток ровно один символ.
- in.getline(char* buf, streamsize limit, char delim = '\n'): считывает в массив char до limit - 1 символов (или до символа-разделителя delim), отбрасывает сам разделитель из потока и автоматически ставит '\0' в конец буфера.
- in.peek(): «подглядывает» следующий символ в потоке и возвращает его, НЕ извлекая его из потока (указатель чтения остаётся на месте).

- in.ignore(streamsize count = 1, int delim = EOF): пропускает и отбрасывает до count символов из входного потока (или до встречи символа delim включительно).
- in.putback(char c) / in.unget(): возвращает последний прочитанный символ обратно во входной поток.

Блочный (побайтовый) неформатированный обмен: read, write и gcount

Для высокоскоростного чтения и записи сырых блоков оперативной памяти (чисел, массивов, структур) без какого-либо текстового форматирования применяются методы read и write:

- ostream& write(const char* buffer, streamsize numBytes): копирует ровно numBytes байт из памяти по адресу buffer напрямую в поток.
- istream& read(char* buffer, streamsize numBytes): считывает до numBytes сырых байт из потока в память по адресу buffer. Так как оба метода принимают указатель типа char*, адрес любой переменной или структуры приводят с помощью reinterpret_cast<char*>(&obj).
- streamsize gcount() const: возвращает количество байт, которые реально удалось прочитать в ходе последнего вызова read(), get() или getline() (очень полезно при чтении последнего неполного блока файла).

Пример кода C++: Работа неформатированных методов get(), peek(), write(), read() и gcount()

Показывает подглядывание следующего символа через peek() и побайтовую запись/чтение массива целых чисел через write() и read().

```
#include <iostream>
#include <sstream>

int main() {
    // 1. Демонстрация peek(), get() и ignore()
    std::istringstream textStream("   #Комментарий
Данные");
    while (textStream.peek() == ' ') {
        textStream.get(); // Пропускаем пробелы по одному
    }
    if (textStream.peek() == '#') {
        textStream.ignore(100, '
'); // Пропускаем всю строку комментария до '
```

```
'
    }
    std::string word;
    textStream >> word;
    std::cout << "Слово после пропуска комментария: " << word << "
";

    // 2. Блочный неформатированный вывод и ввод (write / read / gcount)
    std::stringstream rawStream(std::ios::in | std::ios::out | std::ios:
    int source[3] = {1024, 2048, 4096};

    // Записываем весь массив (12 байт) одним вызовом write
    rawStream.write(reinterpret_cast<const char*>(source), sizeof(source

    int target[3] = {0, 0, 0};
    rawStream.read(reinterpret_cast<char*>(target), sizeof(target));
    std::cout << "Прочитано байт (gcount): " << rawStream.gcount()
              << " -> элементы: " << target[0] << ", " << target[1] << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
Слово после пропуска комментария: Данные
Прочитано байт (gcount): 12 -> элементы: 1024, 2048, 4096
```

Совет для экзамена: На экзамене часто спрашивают разницу между методом потока in.getline(char* buf, int n) и свободной функцией std::getline(istream& in, std::string& s): первый метод является членом класса istream и работает с буфером C-строки фиксированного размера char[], а вторая функция из <string> работает с динамическим классом std::string и сама расширяет строку до любой длины!

36. Текстовые файлы. Форматированный ввод-вывод.

Текстовый файл представляет собой последовательность символов, организованную в строки, разделённые символом перевода строки. При форматированном вводе-выводе все числа и типы данных преобразуются в последовательность ASCII/UTF-8 символов, которые легко просмотреть и отредактировать в любом текстовом редакторе.

Ключевые тезисы:

- В текстовом файле число int x = 1000000 записывается как 7 отдельных символов ('1', '0', '0', '0', '0', '0', '0' — 7 байт), а не как 4 байта машинного кода.
- В текстовом режиме открытия (по умолчанию без ios::binary) на ОС Windows пара символов возврата каретки и перевода строки CR+LF (\r\n) при чтении автоматически преобразуется в один символ \n, а при записи \n превращается обратно в \r\n.
- Для чтения текстовых файлов со сложной структурой удобно сочетать построчное чтение std::getline(file, line) и разбор каждой строки через std::istringstream.

Особенности организации текстовых файлов и трансляции концов строк

Текстовый файл содержит исключительно печатные символы и управляющие коды форматирования (пробел ' ', табуляция '\t', перевод строки '\n'). Главное достоинство текстовых файлов — их кроссплатформенность и наглядность для человека (форматы .txt, .csv, .json, исходный код .cpp).

Важная особенность текстового режима: в разных ОС конец строки кодируется по-разному (в Linux и macOS — один байт LF '\n' с кодом 0x0A; в Windows — два байта CR+LF "\r\n" с кодами 0x0D 0x0A). Когда файл открыт в текстовом режиме

(без флага std::ios::binary), библиотека C++ автоматически переводит "\r\n" в '\n' при чтении и обратно при записи.

Способы чтения и записи текстовых файлов

При работе с текстовыми файлами используют три базовых паттерна чтения:

- 1. Полексемное форматированное чтение через оператор >>: while (in >> word) или while (in >> id >> name >> score). Поток автоматически пропускает любое количество пробелов и переводов строк и парсит данные в переменные нужных типов.
- 2. Построчное чтение: std::string line; while (std::getline(in, line)) { ... } — считывает файл целиком строка за строкой, сохраняя все пробелы внутри строки.
- 3. Посимвольное чтение: char ch; while (in.get(ch)) { ... } — используется для подсчёта точного числа символов, пробелов или шифрования текста.

Пример кода C++: Форматированная запись ведомости в текстовый поток и её парсинг

Запись структурированных текстовых строк и последующее чтение с подсчётом итоговой статистики.

```
#include <iostream>
#include <sstream>
#include <string>
#include <iomanip>

int main() {
    // Используем потоковый буфер для демонстрации текстового файла "rep
    std::stringstream txtFile;

    // 1. Форматированная запись в текстовый файл
    txtFile << "Молоко " << 2 << " " << 85.50 << "
";
    txtFile << "Хлеб "   << 1 << " " << 42.00 << "
";
    txtFile << "Сыр "    << 3 << " " << 310.25 << "
";

    // 2. Форматированное чтение полей из текстового файла
    std::string itemName;
    int quantity = 0;
    double unitPrice = 0.0;
```

```
    double grandTotal = 0.0;

    std::cout << "Чтение позиций из текстового файла:
";
    while (txtFile >> itemName >> quantity >> unitPrice) {
        double lineCost = quantity * unitPrice;
        grandTotal += lineCost;
        std::cout << "  - " << std::left << std::setw(8) << itemName
                  << ": " << quantity << " шт. x "
                  << std::fixed << std::setprecision(2) << unitPrice
                  << " = " << lineCost << " руб.
";
    }
    std::cout << "Итоговая сумма по чеку: " << grandTotal << " руб.
";
    return 0;
}
```

Результат выполнения (stdout):

```
Чтение позиций из текстового файла:
  - Молоко  : 2 шт. x 85.50 = 171.00 руб.
  - Хлеб    : 1 шт. x 42.00 = 42.00 руб.
  - Сыр     : 3 шт. x 310.25 = 930.75 руб.
Итоговая сумма по чеку: 1143.75 руб.
```

Совет для экзамена: Помните: при форматированной записи чисел в текстовый файл между ними ОБЯЗАТЕЛЬНО нужно выводить разделитель (пробел " " или перевод строки "\n")! Если выполнить file << 15 << 20;, в текстовый файл запишется слитная строка "1520", и при чтении file >> а программа прочитает одно число 1520 вместо двух!

37. Бинарные файлы. Последовательный метод доступа.

Бинарный (двоичный) файл хранит данные на диске в том же самом внутреннем машинном (побайтовом) представлении, в каком они располагаются в оперативной памяти. Открывается с обязательным флагом std::ios::binary и обрабатывается методами write() и read().

Ключевые тезисы:

- Преимущества бинарных файлов: высокая скорость чтения/записи (нет затрат на конвертацию чисел в строки), отсутствие потери точности вещественных чисел и фиксированный размер записей.
- Флаг std::ios::binary отключает автоматическую замену символов конца строки (\r\n ↔ \n), чтобы байт 0x0A внутри числа не был искажён.
- В бинарный файл методом write() можно напрямую сохранять примитивные типы, массивы и POD-структуры (без указателей и std::string внутри!).

Отличия бинарных файлов от текстовых и роль флага std::ios::binary

В бинарном файле любая переменная типа int всегда занимает ровно 4 байта (sizeof(int)), а переменная double — ровно 8 байт, независимо от величины числа. Никакого перевода чисел в десятичные ASCII-символы не происходит — байты копируются из ОЗУ в файл один к одному.

Критерий сравнения	Текстовый файл	Бинарный (двоичный) файл
Представление данных	Символьные строки ASCII/UTF-8 (читаемы в Блокноте).	Побайтовые копии ячеек ОЗУ (в текстовом редакторе выглядят как «кракозябры»).

Критерий сравнения	Текстовый файл	Бинарный (двоичный) файл
Скорость и точность	Медленнее из-за парсинга текста; возможна потеря точности double при округлении.	Максимальная скорость копирования памяти; 100% сохранение всех битов мантиссы double.
Размер записи на диске	Переменный (число 1 занимает 1 байт, а 1000000000 — 10 байт).	Строго фиксированный: sizeof(T) байт для любого значения типа T.
Основные операции C++	Операторы <<, >> и getline().	Неформатированные методы write() и read() с флагом std::ios::binary.

Последовательное чтение и запись структур в бинарный файл

При последовательном доступе структуры или элементы массива записываются в бинарный поток подряд друг за другом: out.write(reinterpret_cast<const char*>(&record), sizeof(Record)); и считываются в цикле: while (in.read(reinterpret_cast<char*>(&record), sizeof(Record))) { ... }.

КРИТИЧЕСКОЕ ПРАВИЛО для структур в бинарных файлах: напрямую записывать через write(&s, sizeof(s)) можно только плоские POD-структуры (Plain Old Data), содержащие примитивные поля и статические массивы фиксированной длины (например, char name[32];). Если структура содержит std::string или std::vector, то внутри неё хранится лишь указатель на кучу! Запись такой структуры через sizeof сохранит в файл бесполезный адрес памяти вместо самого текста.

Пример кода C++: Последовательная запись и чтение массива структур в бинарном потоке

Демонстрирует сохранение записей фиксированного размера в двоичном режиме и их последовательное чтение до конца потока.

```
#include <iostream>
#include <sstream>
#include <cstring>
```

```
// POD-структура фиксированного размера (безопасна для бинарной записи)
struct SensorRecord {
    int sensorId;
    char code[16]; // Фиксированный массив символов вместо std::string
    double reading;
};

int main() {
    // Открываем поток в бинарном режиме (std::ios::binary)
    std::stringstream binFile(std::ios::in | std::ios::out | std::ios::b

    SensorRecord outData[2] = {
        {101, "TEMP_NORTH", 23.456789},
        {102, "PRESS_MAIN", 1013.25}
    };

    // 1. Последовательная запись структур в бинарный файл
    for (int i = 0; i < 2; ++i) {
        binFile.write(reinterpret_cast<const char*>(&outData[i]), sizeof
    }

    // 2. Последовательное чтение структур из бинарного файла
    SensorRecord rec{};
    std::cout << "Размер одной структуры SensorRecord: " << sizeof(Senso
";
    while (binFile.read(reinterpret_cast<char*>(&rec), sizeof(SensorReco
        std::cout << "Датчик #" << rec.sensorId << " [" << rec.code
                  << "] = " << rec.reading << "
";
    }
    return 0;
}
```

Результат выполнения (stdout):

```
Размер одной структуры SensorRecord: 32 байт
Датчик #101 [TEMP_NORTH] = 23.4568
Датчик #102 [PRESS_MAIN] = 1013.25
```

Совет для экзамена: Если на экзамене нужно записать std::string s в бинарный файл, нельзя писать file.write((char*)&s, sizeof(s))! Правильный способ из 2 шагов: 1) сначала записать длину строки size_t len = s.size(); file.write((char*)&len, sizeof(len));, 2) затем записать сами символы буфера file.write(s.data(), len);.

38. Организация прямого доступа к файлу.

Прямой (произвольный — Random Access) доступ к файлу позволяет мгновенно перемещать указатель чтения (get pointer) или записи (put pointer) в любую точку файла без последовательного перебора предшествующих байтов, используя методы seekg, seekp, tellg и tellp.

Ключевые тезисы:

- Методы с суффиксом g (get — чтение) работают во входных потоках: tellg() и seekg(). Методы с суффиксом p (put — запись) работают в выходных потоках: tellp() и seekp().
- Три базовые точки отсчёта смещения: std::ios::beg (от начала файла), std::ios::cur (от текущей позиции) и std::ios::end (от конца файла).
- В бинарном файле из однотипных записей типа Record смещение k-й записи (нумерация с 0) вычисляется за O(1) по формуле: k * sizeof(Record).

Позиционирование указателей в потоке: seekg, seekp, tellg, tellp

Файловая система ОС рассматривает любой файл как большой одномерный массив байтов, пронумерованных с 0. В потоках C++ поддерживаются два логических указателя позиции: указатель чтения (get pointer) и указатель записи (put pointer).

Метод потока	Назначение	Примеры использования
pos_type tellg() / tellp()	Возвращает текущую позицию (смещение в байтах от начала файла) указателя чтения / записи.	Используется для запоминания позиции или определения общего размера файла.
seekg(pos) / seekp(pos)	Абсолютное позиционирование: перемещает указатель чтения / записи на байт с индексом pos от начала файла.	file.seekg(0); — перемотка в самое начало файла.

Метод потока	Назначение	Примеры использования
seekg(offset, dir) / seekp(offset, dir)	Относительное позиционирование: сдвигает указатель на offset байт (может быть отрицательным!) относительно точки отсчёта dir.	dir может принимать три значения: std::ios::beg (начало), std::ios::cur (текущая позиция), std::ios::end (конец файла).

Практические приёмы прямого доступа в бинарных базах данных

Прямой доступ раскрывает всю свою мощь при работе с бинарными файлами, содержащими однотипные структуры фиксированного размера sizeof(Record):

- 1. Определение размера файла в байтах и числа записей: вызываем file.seekg(0, std::ios::end); auto fileSize = file.tellg(); size_t count = fileSize / sizeof(Record);.
- 2. Мгновенное чтение k-й записи (при индексации с 0): file.seekg(k * sizeof(Record), std::ios::beg); file.read(reinterpret_cast<char*>(&rec), sizeof(Record)); — выполняется за константное время O(1) даже в файле на 10 гигабайт!
- 3. Модификация (перезапись) одной записи «на месте» (in-place update): открываем поток std::fstream с флагами ios::in | ios::out | ios::binary, сдвигаем указатель записи на k * sizeof(Record) через seekp и перезаписываем ровно одну структуру методом write.

Пример кода C++: Прямой доступ к бинарному файлу: чтение по индексу и перезапись элемента на месте

Показывает определение размера файла через seekg(0, ios::end) + tellg(), чтение 3-го элемента за O(1) и перезапись 1-го элемента через seekp().

```
#include <iostream>
#include <sstream>

int main() {
    std::stringstream db(std::ios::in | std::ios::out | std::ios::binary

    // Записываем массив из 5 целых чисел: 10, 20, 30, 40, 50
    int initial[5] = {10, 20, 30, 40, 50};
    db.write(reinterpret_cast<const char*>(initial), sizeof(initial));
```

```
    // 1. Узнаём размер файла в байтах через прямой доступ к концу файла
    db.seekg(0, std::ios::end);
    std::streamoff totalBytes = db.tellg();
    std::cout << "Размер файла: " << totalBytes << " байт ("
              << (totalBytes / sizeof(int)) << " элементов int)
";

    // 2. Прямое чтение элемента с индексом 3 (число 40) без чтения пред
    int valAt3 = 0;
    db.seekg(3 * sizeof(int), std::ios::beg);
    db.read(reinterpret_cast<char*>(&valAt3), sizeof(int));
    std::cout << "Прямое чтение индекса [3]: " << valAt3 << "
";

    // 3. Прямая перезапись элемента с индексом 1 (заменяем 20 на 999)
    int newValue = 999;
    db.seekp(1 * sizeof(int), std::ios::beg);
    db.write(reinterpret_cast<const char*>(&newValue), sizeof(int));

    // Проверяем весь файл после точечной модификации
    db.seekg(0, std::ios::beg);
    std::cout << "Файл после модификации [1]: ";
    int x = 0;
    while (db.read(reinterpret_cast<char*>(&x), sizeof(int))) {
        std::cout << x << " ";
    }
    std::cout << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
Размер файла: 20 байт (5 элементов int)
Прямое чтение индекса [3]: 40
Файл после модификации [1]: 10 999 30 40 50
```

Совет для экзамена: Важный нюанс на экзамене: если до вызова seekg() вы дочитали файл до самого конца и сработал флаг eofbit, то в C++ вызов seekg() НЕ сбрасывает флаги ошибок! Обязательно сначала вызовите file.clear();, и только после этого делайте file.seekg(0, std::ios::beg);.

39. Исключительные ситуации. Классификация исключительных ситуаций. Операторы для работы с исключениями. Защищённые блоки. Выброс системных и пользовательских исключений. Последовательность действий при обработке исключений. Необработанные исключения. Собственная функция аварийного завершения.

Механизм обработки исключений в C++ (try, throw, catch) позволяет отделить код обнаружения аварийных ситуаций от кода их обработки, обеспечивая автоматическую раскрутку стека (Stack Unwinding) с вызовом деструкторов всех локальных объектов.

Ключевые тезисы:

- Три ключевых слова: try (защищённый блок), throw (генерация/выброс исключения) и catch (перехват и обработка исключения по типу).
- Стандартная иерархия исключений в <stdexcept> наследуется от базового класса std::exception с виртуальным методом what(), который возвращает текстовое описание ошибки.
- При раскрутке стека (Stack Unwinding) автоматически вызываются деструкторы всех созданных в блоке try локальных объектов. Если исключение нигде не перехвачено, вызывается std::terminate() (настраивается через std::set_terminate).

Понятие и классификация исключительных ситуаций в C++

Исключительная ситуация (Exception) — это событие или ошибка времени выполнения, которая делает невозможным нормальное продолжение работы

текущей функции (например, деление на ноль, нехватка памяти в new, отсутствие файла, выход индекса за границы в vector::at).

В стандартной библиотеке C++ (заголовки <exception> и <stdexcept>) все стандартные исключения образуют иерархию с корнем в базовом классе std::exception (имеет метод const char* what() const noexcept):

Ветвь иерархии	Конкретные классы исключений	Когда выбрасываются
Логические ошибки (std::logic_error)	std::invalid_argument, std::out_of_range, std::domain_error, std::length_error	Ошибки в логике вызова API (нарушение предусловий), например: выход за границы в std::vector::at(i) или некорректная строка в std::stoi("abc").
Ошибки времени выполнения (std::runtime_error)	std::range_error, std::overflow_error, std::underflow_error, std::system_error	Ошибки, зависящие от внешних факторов или переполнения при вычислениях в рантайме.
Системные исключения языка и памяти	std::bad_alloc, std::bad_cast, std::bad_typeid	Генерируются самими операторами C++: например, оператор new выбрасывает std::bad_alloc при исчерпании памяти в куче.
Пользовательские исключения	Собственные структуры/классы (рекомендуется наследовать от std::exception или std::runtime_error)	Выбрасываются программистом через throw при нарушении бизнес-правил приложения (например, InsufficientFundsException).

Защищённые блоки (try-catch), раскрутка стека и необработанные исключения

Механизм работы с исключениями опирается на три конструкции: 1) Защищённый блок try { ... } — оборачивает код, в котором потенциально может возникнуть ошибка; 2) Инструкция выброса throw expr; — создаёт объект исключения и немедленно прерывает выполнение текущей функции; 3) Обработчики catch (const

ТипИсключения& e) { ... } — идут сразу после блока try и проверяются сверху вниз на соответствие типу выброшенного объекта.

- Последовательность действий при выбросе исключения (Раскрутка стека — Stack Unwinding): как только выполняется throw, управление покидает текущий блок. Если в текущей функции нет подходящего catch, функция принудительно завершается, и поиск обработчика поднимается вверх по стеку вызовов в вызывающую функцию. При этом C++ ГАРАНТИРУЕТ вызов деструкторов для всех локальных объектов с автоматическим временем жизни, созданных в покидаемых блоках try (идиома RAII)!
- Правила порядка блоков catch: обработчики просматриваются строго сверху вниз. Поэтому catch для более конкретных (производных) классов (например, const std::out_of_range&) ОБЯЗАНЫ стоять ВЫШЕ обработчика базового класса (const std::exception&). В самом конце можно поставить универсальный перехватчик с троеточием catch (...) { ... }, который ловит абсолютно любые типы исключений.
- Необработанные исключения и функция аварийного завершения: если исключение прошло весь стек вызовов вплоть до main() и не нашло ни одного подходящего блока catch (или если новое исключение было выброшено из деструктора во время раскрутки стека), оно считается необработанным. В этот момент среда выполнения вызывает функцию std::terminate() (по умолчанию она вызывает std::abort()). Программист может установить собственную функцию аварийного завершения с помощью std::set_terminate(myHandler) из заголовка <exception>.

Пример кода C++: Пользовательское исключение, иерархия catch и установка собственной функции std::set_terminate

Показывает выброс и перехват стандартных и пользовательских исключений, а также регистрацию собственного обработчика аварийного завершения.

```
#include <iostream>
#include <stdexcept>
#include <exception>
#include <vector>

// 1. Пользовательский класс исключения, унаследованный от std::runtime_
class DivisionByZeroError : public std::runtime_error {
public:
    DivisionByZeroError()
        : std::runtime_error("Математическая ошибка: попытка деления на
```

```
};

// 2. Собственная функция аварийного завершения (для необработанных искл
void customTerminateHandler() {
    std::cerr << "[CRITICAL] Перехвачено необработанное исключение! Заве
";
    std::abort();
}

double safeDivide(double a, double b) {
    if (b == 0.0) {
        throw DivisionByZeroError(); // Выброс пользовательского исключе
    }
    return a / b;
}

int main() {
    // Регистрируем собственную функцию аварийного завершения
    std::set_terminate(customTerminateHandler);

    // Защищённый блок try
    try {
        std::vector<int> v = {10, 20, 30};
        std::cout << "Деление 100 / 4 = " << safeDivide(100.0, 4.0) << "
";

        // Генерируем исключение деления на ноль
        std::cout << safeDivide(50.0, 0.0) << "
";

        // Эта строка уже не выполнится — управление перейдёт в блок cat
        std::cout << v.at(10) << "
";
    }
    // Сначала ловим конкретные производные исключения по константной со
    catch (const DivisionByZeroError& e) {
        std::cout << "Перехвачено пользовательское исключение: " << e.wh
";
    }
    catch (const std::out_of_range& e) {
        std::cout << "Выход за границы контейнера: " << e.what() << "
";
    }
    // Затем базовый класс стандартных исключений:
    catch (const std::exception& e) {
        std::cout << "Стандартное исключение: " << e.what() << "
";
    }
    // В самом конце — универсальный перехватчик любых других типов:
```

```
    catch (...) {
        std::cout << "Перехвачено неизвестное исключение!
";
    }

    std::cout << "Программа штатно продолжает работу после блока try-cat
";
    return 0;
}
```

Результат выполнения (stdout):

```
Деление 100 / 4 = 25
Перехвачено пользовательское исключение: Математическая ошибка: попытка
Программа штатно продолжает работу после блока try-catch.
```

Совет для экзамена: На экзамене всегда подчёркивайте: выбрасывать исключения нужно по значению (throw MyException("...");), а перехватывать в блоке catch — по константной ссылке (catch (const std::exception& e)), чтобы избежать лишнего копирования и «срезки» полиморфного объекта (Object Slicing)!